

# Network Computing

(058172)



**POLITECNICO**  
MILANO 1863

# Datacenter Networking

Some parts of the slides are inspired from Mohammad Alizadeh's and Sylvia Ratnasamy's and Marco Chiesa lectures on datacenter networks

# Content

- What is a datacenter?  
(differences between other traditional networks, e.g., WANs)

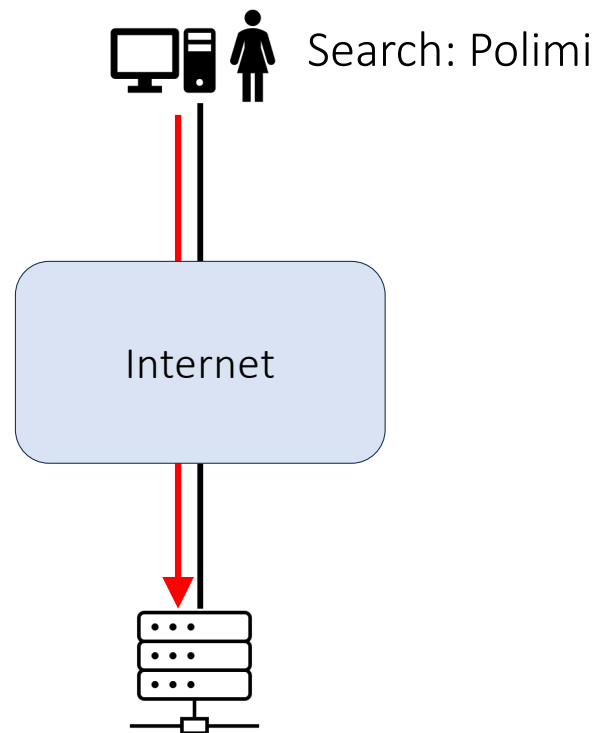
# Content

- What is a datacenter?  
(differences between other traditional networks, e.g., WANs)
- Why are datacenters important?  
(what type of applications do we run on DCs)

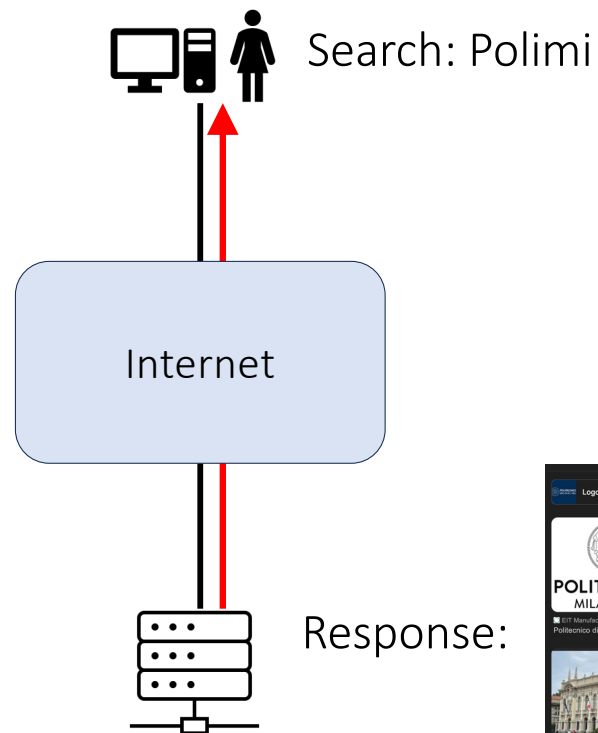
# Content

- What is a datacenter?  
(differences between other traditional networks, e.g., WANs)
- Why are datacenters important?  
(what type of applications do we run on DCs)
- What are the opportunities/challenges?  
(traditional L2/L3 routing/congestion control is suboptimal)

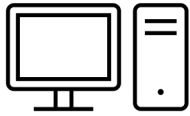
# The web search example: fetching information



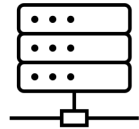
# The web search example: fetching information



# What if a machine is not sufficient?



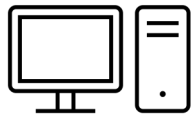
PC



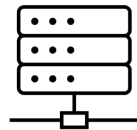
Server



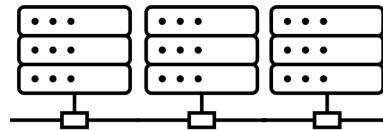
What if a machine is not sufficient? USE MANY!



PC

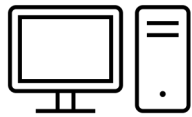


Server

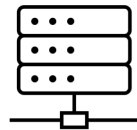


Cluster

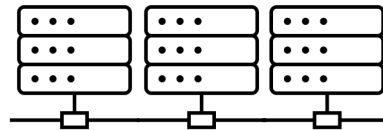
What if a machine is not sufficient? USE MANY!



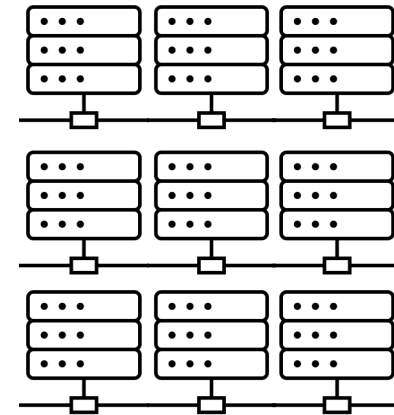
PC



Server



Cluster



Datacenter

# So what's a datacenter?

Datacenters are buildings where *multiple servers and communication equipment are co-located* because of their common environmental requirements and physical security needs

# So what's a datacenter?

Datacenters are buildings where *multiple servers and communication equipment are co-located* because of their common environmental requirements and physical security needs

- *Single* administrative domain
- control over network and endpoint(s)
- control over placement of traffic source/sink

# So what's a datacenter?

Datacenters are buildings where *multiple servers and communication equipment are co-located* because of their common environmental requirements and physical security needs

- *Single* administrative domain
- control over network and endpoint(s)
- control over placement of traffic source/sink

## *Full control:*

- can deviate from standards (invent your own!)
- can change addressing/congestion control
- can control what traffic crosses which links

# Datacenters have been around for a while now..



1961, Information Processing Center at the National Bank of Arizona



# Datacenters have been around for a while now..

Similar to IBM mainframes from the 70s

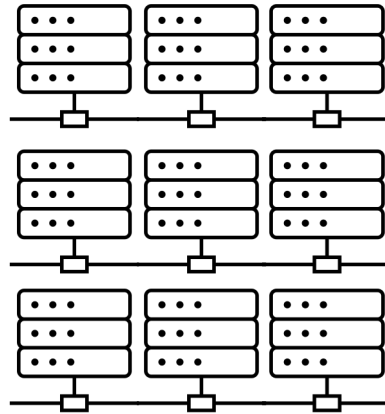


So what's new?



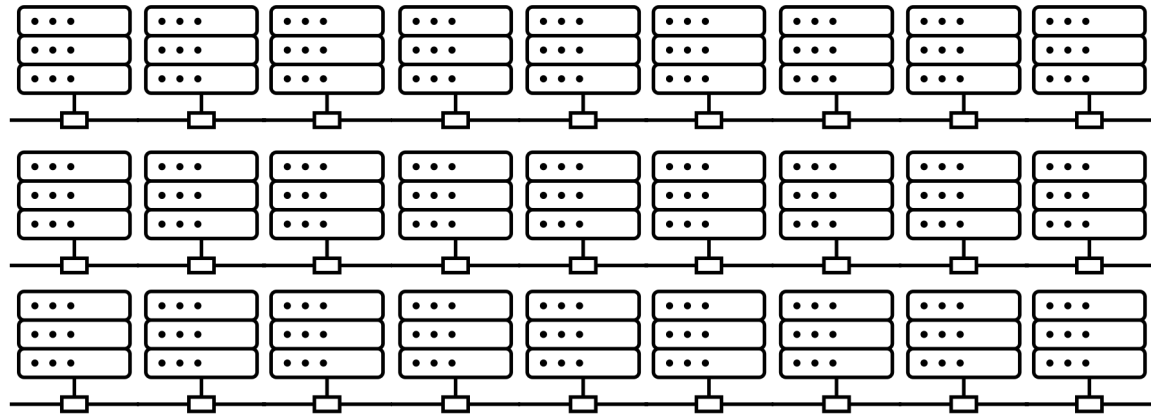
# So what's new?

- Scale



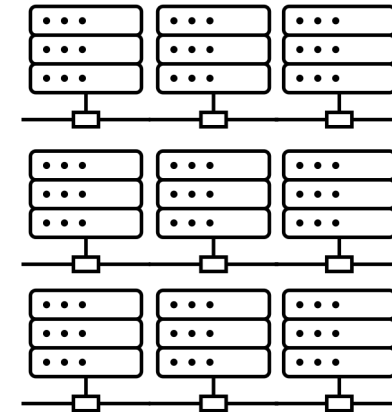
# So what's new?

- Huge Scale!



# So what's new?

- Massive Scale!
  - 1M servers/site [Microsoft/Amazon/Google]
  - 100K switches/routers [Microsoft]
  - Google is serving billion of queries per day
  - Facebook/Instagram count billion of active users



## MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs

Ziheng Jiang<sup>1,\*</sup> Haibin Lin<sup>1,\*</sup> Yinmin Zhong<sup>2,\*</sup> Qi Huang<sup>1</sup> Yangrui Chen<sup>1</sup> Zhi Zhang<sup>1</sup>  
 Yanghua Peng<sup>1</sup> Xiang Li<sup>1</sup> Cong Xie<sup>1</sup> Shibiao Nong<sup>1</sup> Yulu Jia<sup>1</sup> Sun He<sup>1</sup> Hongmin Chen<sup>1</sup>  
 Zhihao Bai<sup>1</sup> Qi Hou<sup>1</sup> Shipeng Yan<sup>1</sup> Ding Zhou<sup>1</sup> Yiyao Sheng<sup>1</sup> Zhuo Jiang<sup>1</sup>  
 Haohan Xu<sup>1</sup> Haoran Wei<sup>1</sup> Zhang Zhang<sup>1</sup> Pengfei Nie<sup>1</sup> Leqi Zou<sup>1</sup> Sida Zhao<sup>1</sup>  
 Liang Xiang<sup>1</sup> Zherui Liu<sup>1</sup> Zhe Li<sup>1</sup> Xiaoying Jia<sup>1</sup> Jianxi Ye<sup>1</sup> Xin Jin<sup>2,†</sup> Xin Liu<sup>1,†</sup>

<sup>1</sup>ByteDance <sup>2</sup>Peking University

### Abstract

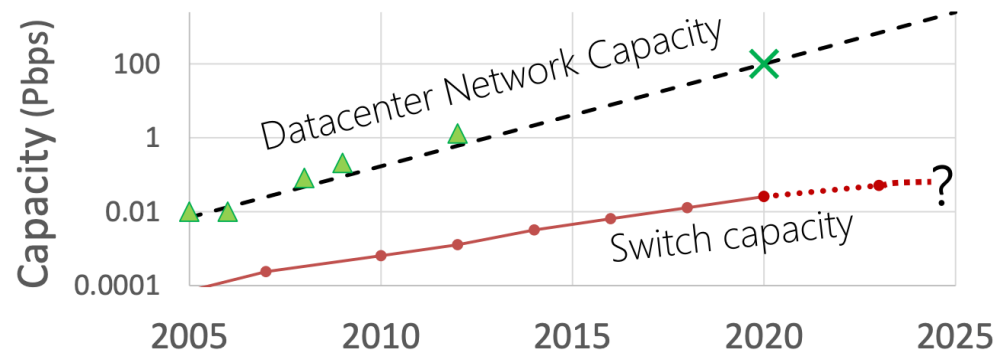
We present the design, implementation and engineering experience in building and deploying MegaScale, a production system for training large language models (LLMs) at the scale of more than 10,000 GPUs. Training LLMs at this scale brings unprecedented challenges to training efficiency and stability. We take a full-stack approach that co-designs the algorithmic and system components across model block and optimizer design, computation and communication overlapping, operator optimization, data pipeline, and network performance tuning. Maintaining high efficiency throughout the training process (i.e., stability) is an important consideration in production given the long extent of LLM training jobs. Many hard stability issues only emerge at large scale, and in-depth observability is the key to address them. We develop a set of diagnosis tools to monitor system components and events deep in the stack, identify root causes, and derive effective techniques to achieve fault tolerance and mitigate stragglers. MegaScale achieves 55.2% Model FLOPs Utilization (MFU) when training a 175B LLM model on 12,288 GPUs, improving the MFU by 1.34x compared to Megatron-LM. We share our operational experience in identifying and fixing failures and stragglers. We hope by articulating the problems and sharing our experience from a systems perspective, this work can inspire future LLM systems research.

serving billions of users, we have been aggressively integrating AI into our products, and we are putting LLMs as a high priority to shape the future of our products.

Training LLMs is a daunting task that requires enormous computation resources. The scaling law [3] dictates that the model size and the training data size are critical factors that determine the model capability. To achieve state-of-the-art model capability, many efforts have been devoted to train large models with hundreds of billions or even trillions of parameters on hundreds of billions or even trillions of tokens. For example, GPT-3 [4] has 175 billion parameters and PaLM [5] has 540 billion parameters. Major players in this field build large-scale AI clusters with tens of thousands of GPUs to train LLMs.

Scaling LLM training to tens of thousands of GPUs brings unprecedented challenges. As AI has been at the core of many of our products, we have extensive experience in training deep neural networks (DNNs). Yet, training a model like ResNet [6] only takes tens or hundreds of GPUs. Compared to these models, the scale of training LLMs is unparalleled. While we are not new to building and operating large-scale GPU clusters, these clusters are normally shared by many training jobs. Now, in the context of LLM training, a single job is occupying tens of thousands of GPUs and taking all the resources. The sheer scale of LLM training introduces two specific challenges from a systems perspective.

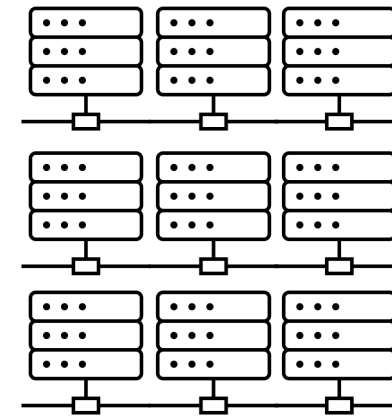
The first challenge is to achieve high training efficiency



Sirius: A Flat Datacenter Network with Nanosecond Optical Switching (ACM SIGCOMM)

# So what's new?

- Massive Scale!
  - 1M servers/site [Microsoft/Amazon/Google]
  - 100K switches/routers [Microsoft]
  - Google is serving billion of queries per day
  - Facebook/Instagram count billion of active users

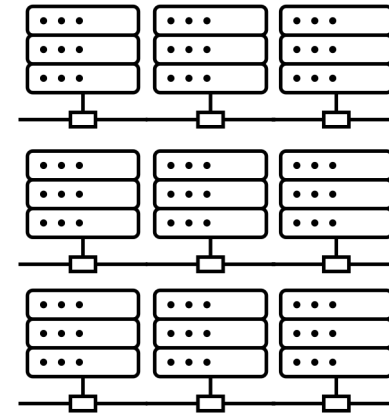


## • Implications

- Need scalable designs ☺
- Try to contain as much as possible the costs
- Try to utilize as much as possible the available resources
- Design solutions that can tolerate failures
- Automate as much as possible

# So what's new?

- Massive Scale!
  - 1M servers/site [Microsoft/Amazon/Google]
  - 100K switches/routers [Microsoft]
  - Google is serving billion of queries per day
  - Facebook/Instagram count billion of active users



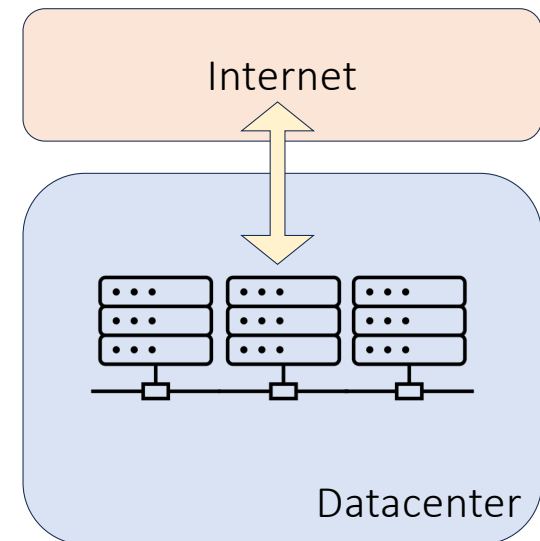
## • Implications

- Need scalable designs ☺
- Try to contain as much as possible the costs
- Try to utilize as much as possible the available resources
- Design solutions that can tolerate failures
- Automate as much as possible

*"In such large systems, software and hardware failures are the norm rather than the exception"*  
[Microsoft Paper from ACM SIGCOMM 2015]

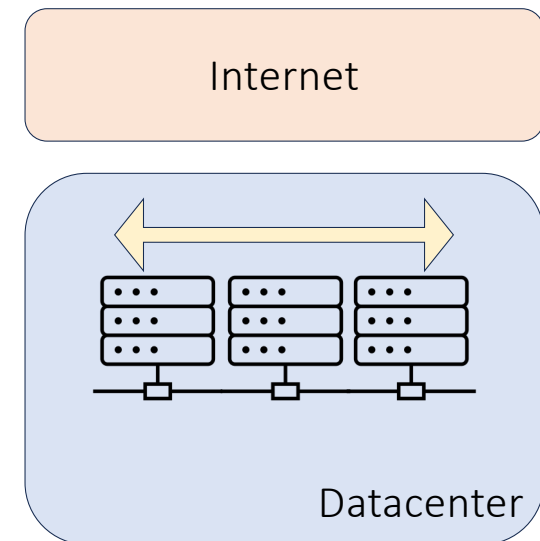
# Applications

- Customer facing traffic
  - Interactive
  - Query-Response exchange between external clients and the datacenter (e.g., serving web-pages to users)
  - Called “North-South” traffic

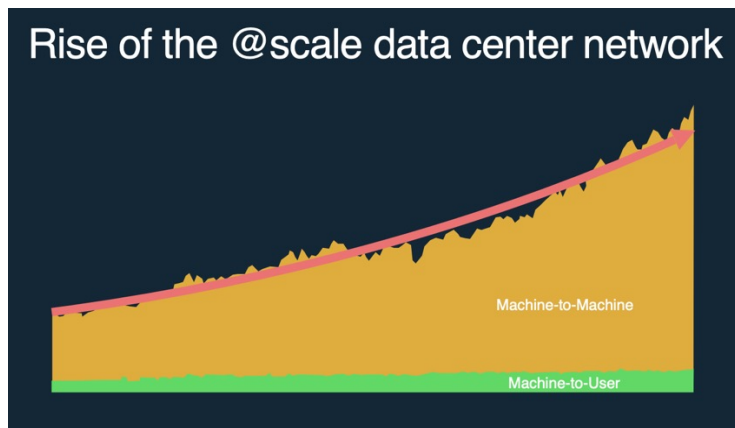


# Applications

- Customer facing traffic
  - Interactive
  - Query-Response exchange between external clients and the datacenter (e.g., serving web-pages to users)
  - Called “North-South” traffic
- Large scale computation
  - Usually for data analysis (e.g., web indexing) or ML training
  - Traffic between servers in the datacenter
  - Called “East-West” traffic

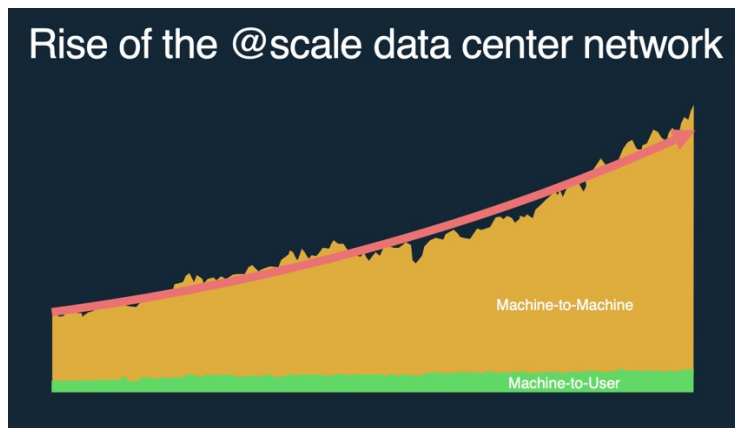


# Interestingly..





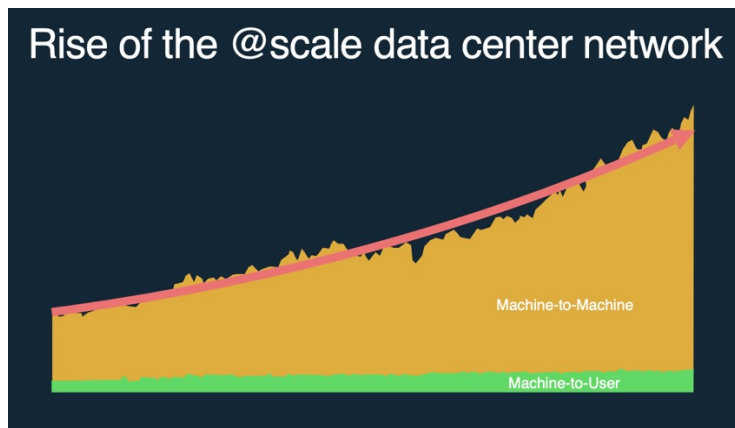
# Interestingly..



## Cloud workloads

- Applications are split in microservices
- Databases and caches are distributed
- User requests generate many internal service calls

# Interestingly..



## Cloud workloads

- Applications are split in microservices
- Databases and caches are distributed
- User requests generate many internal service calls

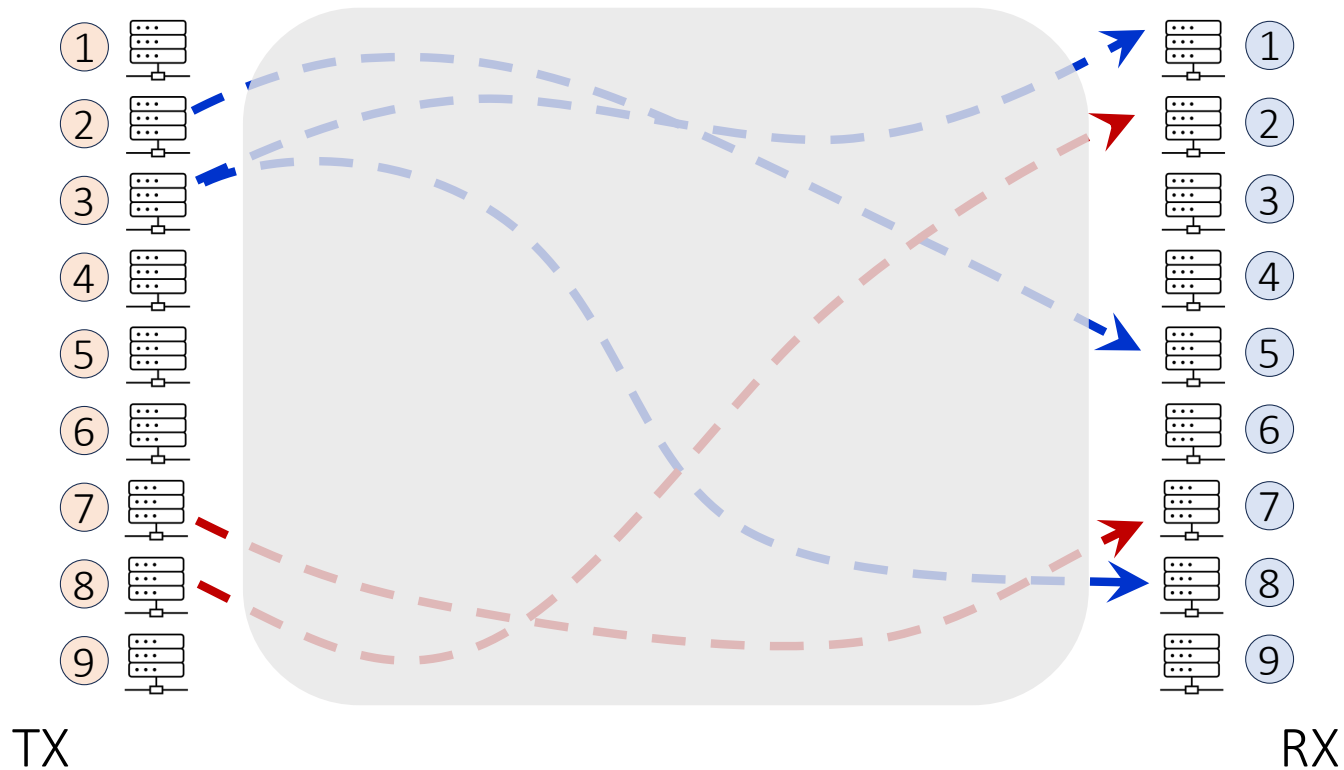
## AI workloads

- Models are too large for a single machine
- Training uses many GPUs across servers
- Communication happens every training iteration

# Implications

- Any-to-any communication
  - high bandwidth is desirable
  - Ideally: each server shall be able to talk to any other server at its full access link rate
- Latency is critical too
  - The performance of applications such as distributed machine learning is sensitive to that
  - In a distributed computation environment, one server can slow down the entire process
- Worst-case (tail) latency is very important

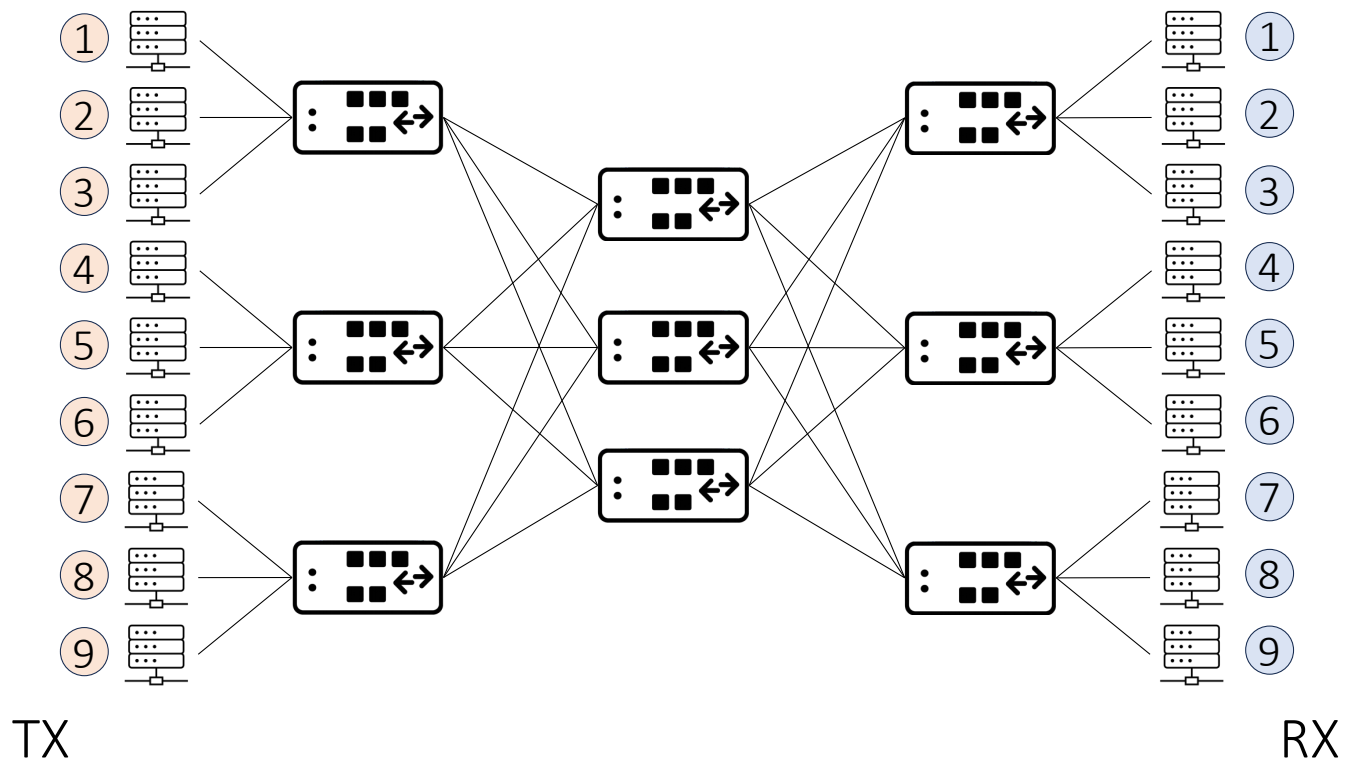
# Datacenter network is “just” a giant switch!



# This is ideally...

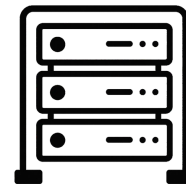
- If it was really one giant switch..
  - It would need to offer around 1M ports (one port/server) at around 100Gbps each (or more)!
  - Very high aggregate throughput! 😊
- Practical approach: build a network of switches (“fabric”)
  - Each switch has practical #ports and link speeds

So how a datacenter network looks like?



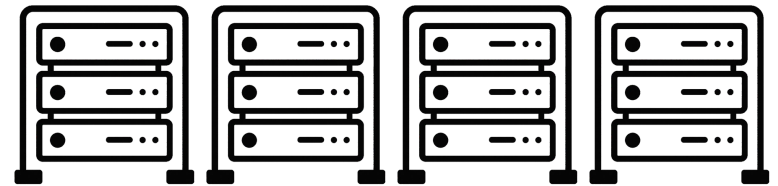
# Datacenter architecture

- 100.000s servers (placed in racks)



# Datacenter architecture

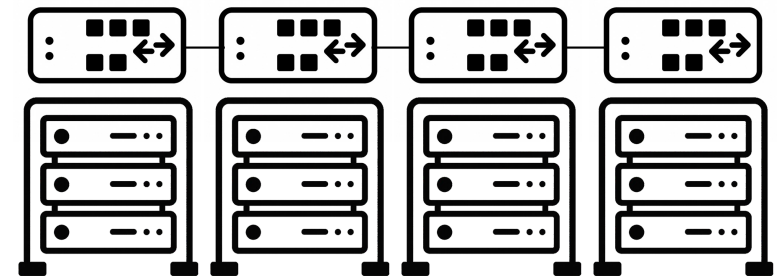
- 100.000s servers (placed in racks)  
interconnected by...





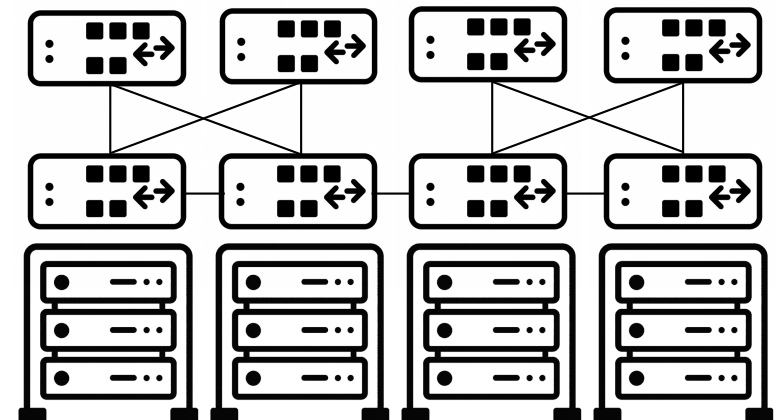
# Datacenter architecture

- 100.000s servers (placed in racks)  
interconnected by...
- 1000s top-of-rack (ToR) switches



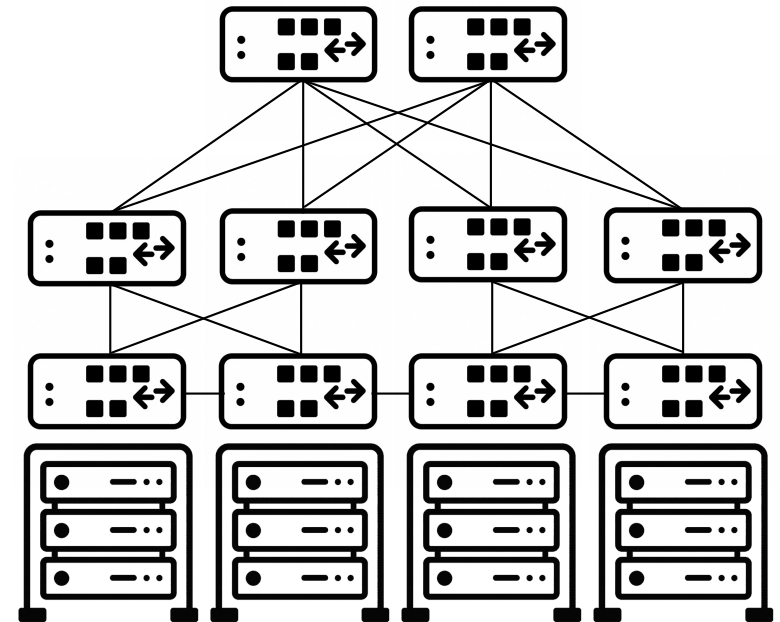
# Datacenter architecture

- 100.000s servers (placed in racks)  
interconnected by...
- 1000s top-of-rack (ToR) switches  
interconnected by...
- 1000s aggregation switches  
interconnected by...



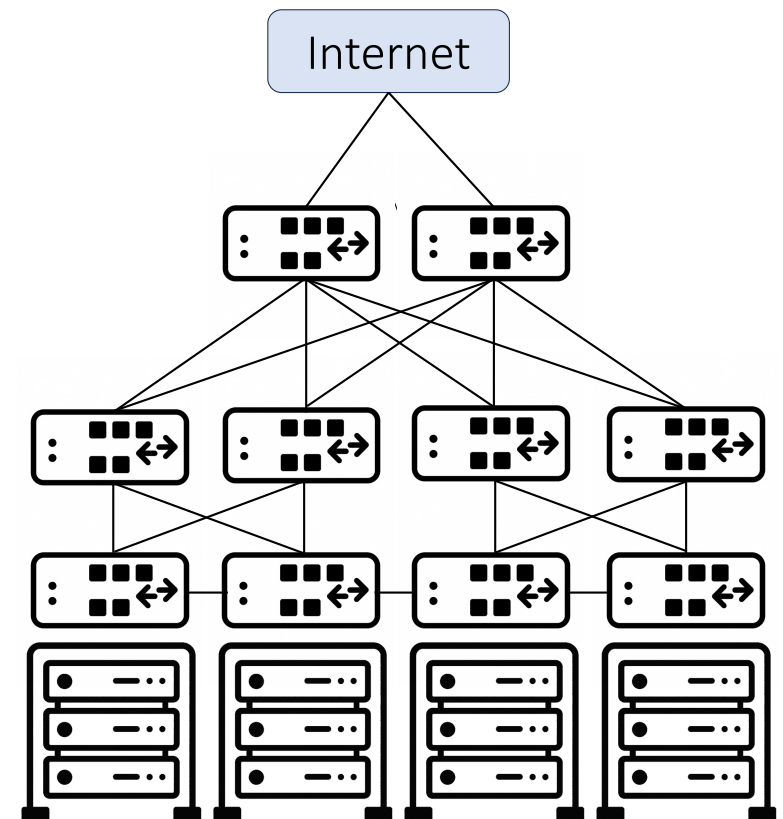
# Datacenter architecture

- 100.000s servers (placed in racks)  
interconnected by...
- 1000s top-of-rack (ToR) switches  
interconnected by...
- 1000s aggregation switches  
interconnected by...
- 100s core switches  
interconnected to...

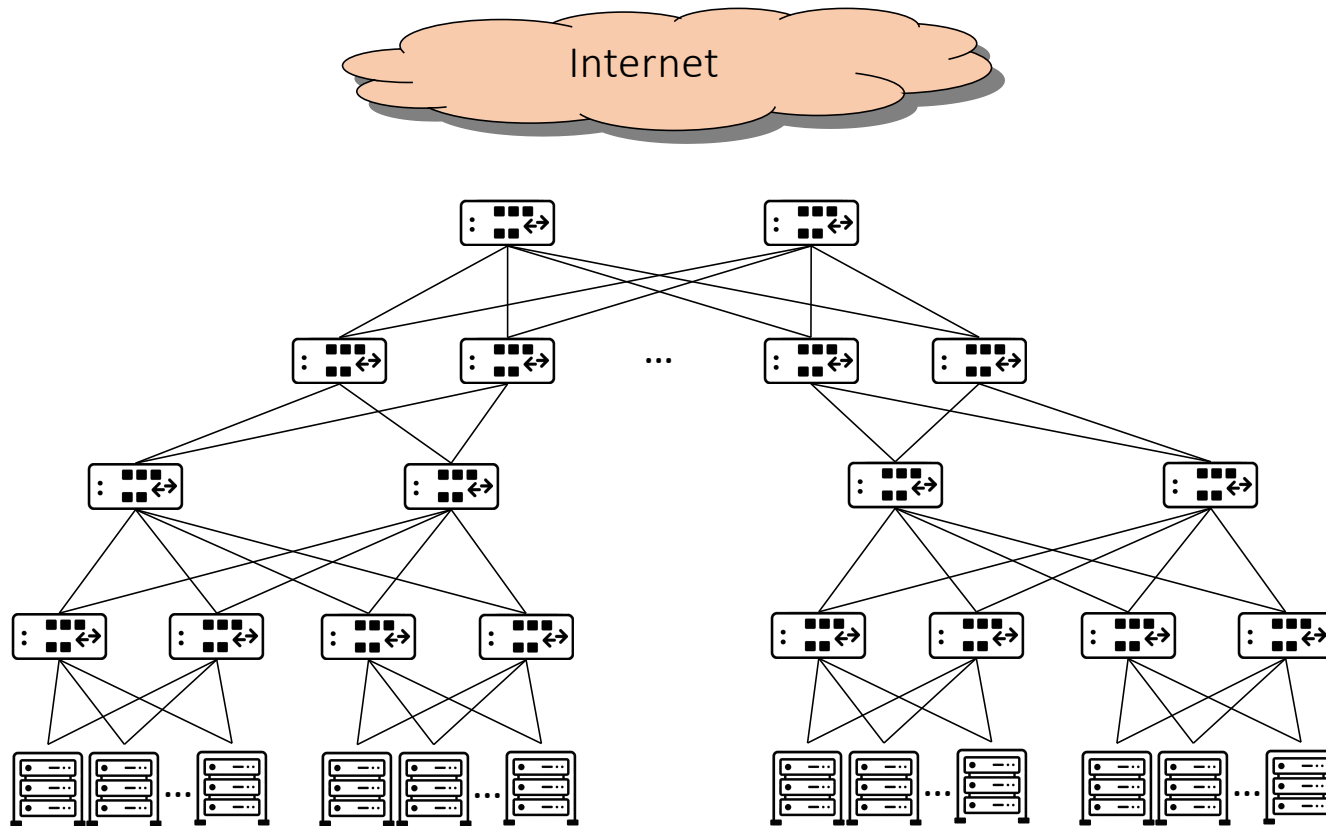


# Datacenter architecture

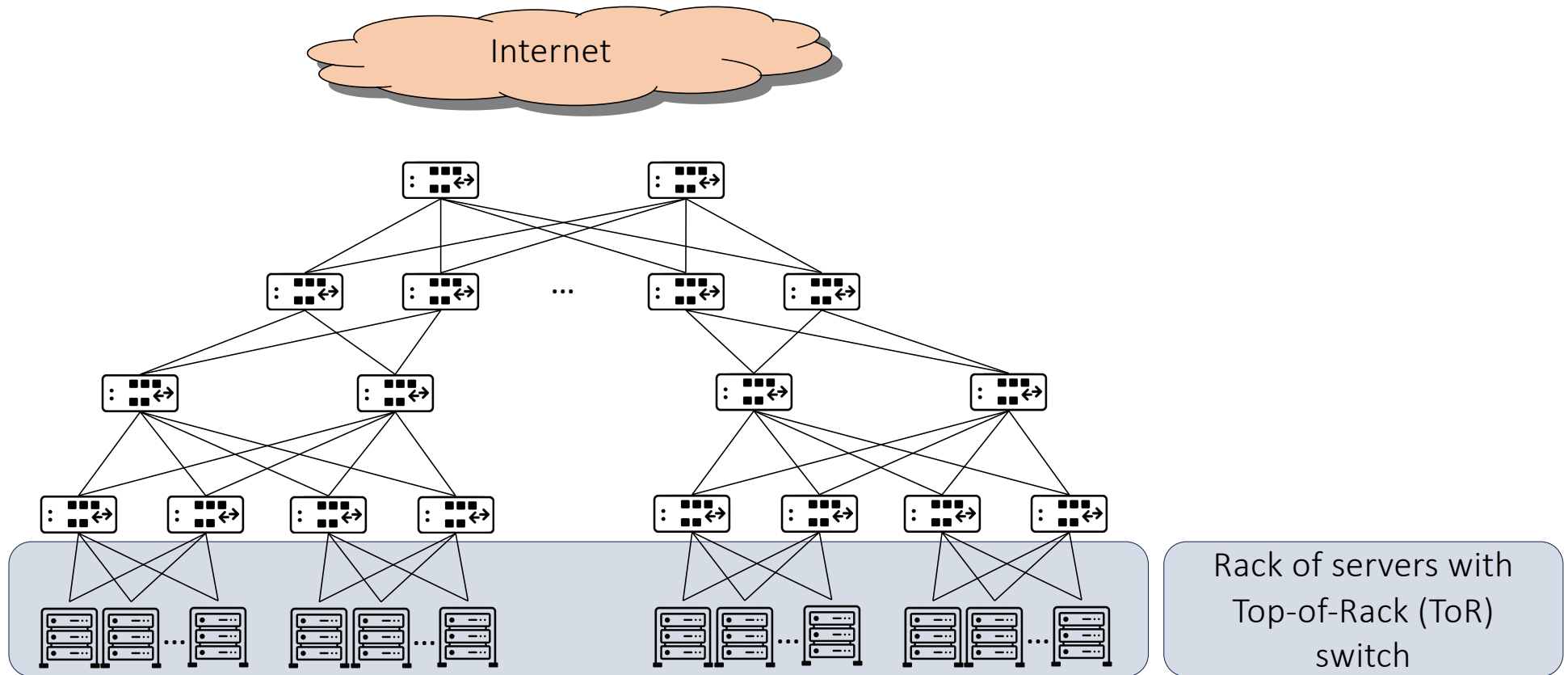
- 100.000s servers (placed in racks)  
interconnected by...
- 1000s top-of-rack (ToR) switches  
interconnected by...
- 1000s aggregation switches  
interconnected by...
- 100s core switches  
interconnected to...  
the Internet/backbone network



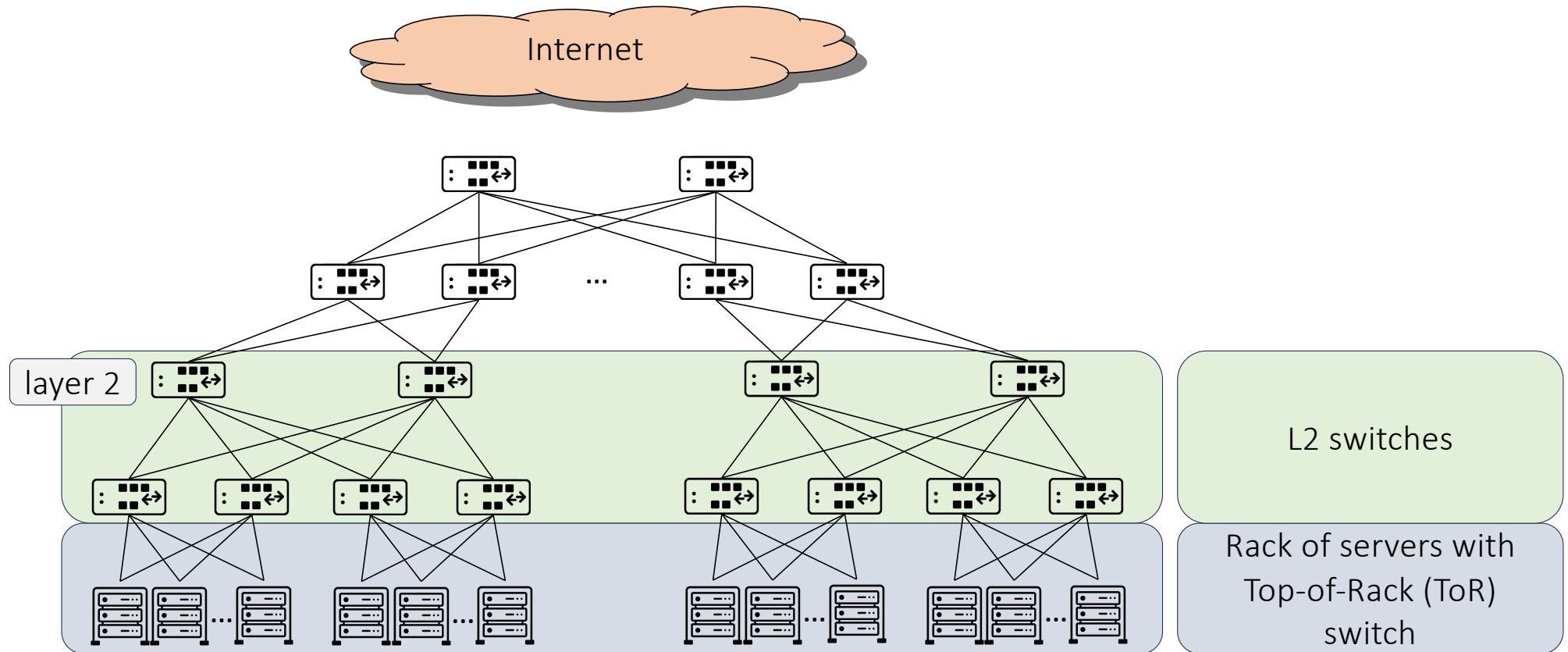
# Conventional Datacenter Network



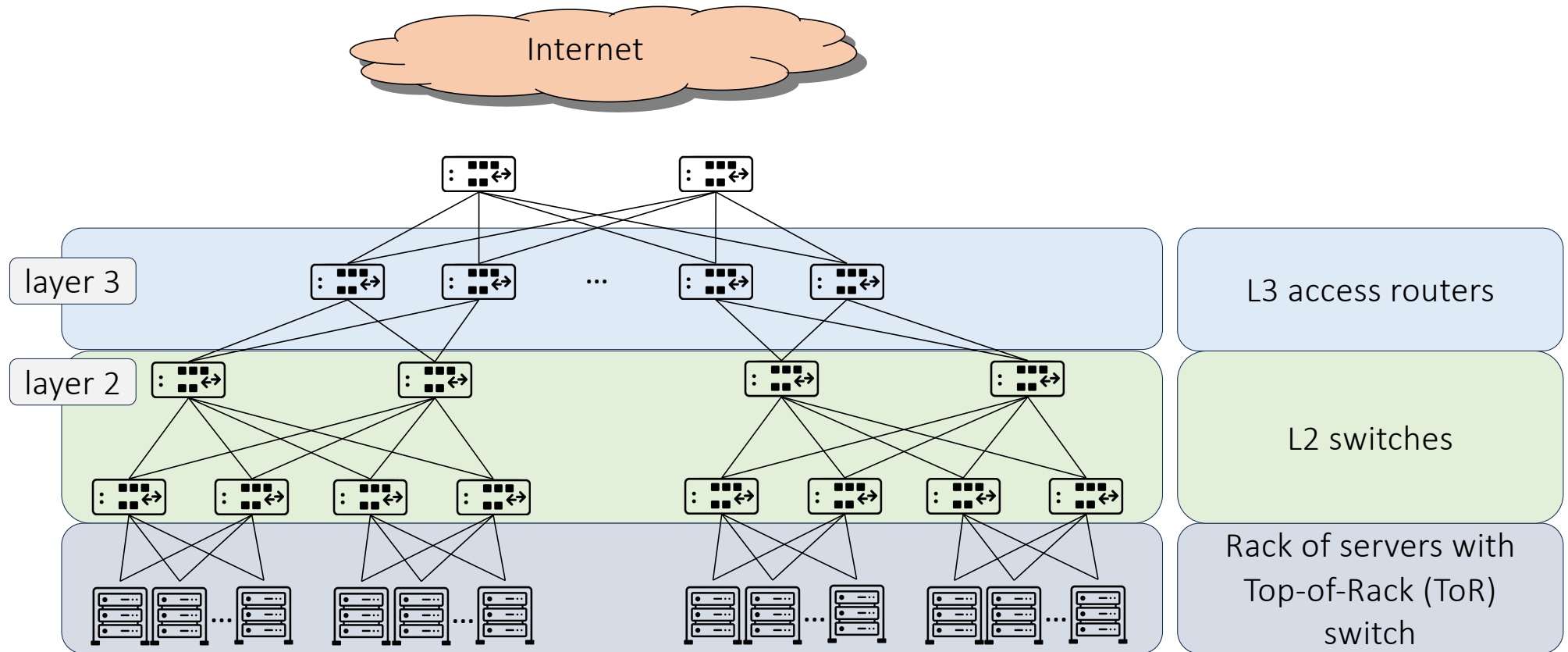
# Conventional Datacenter Network



# Conventional Datacenter Network

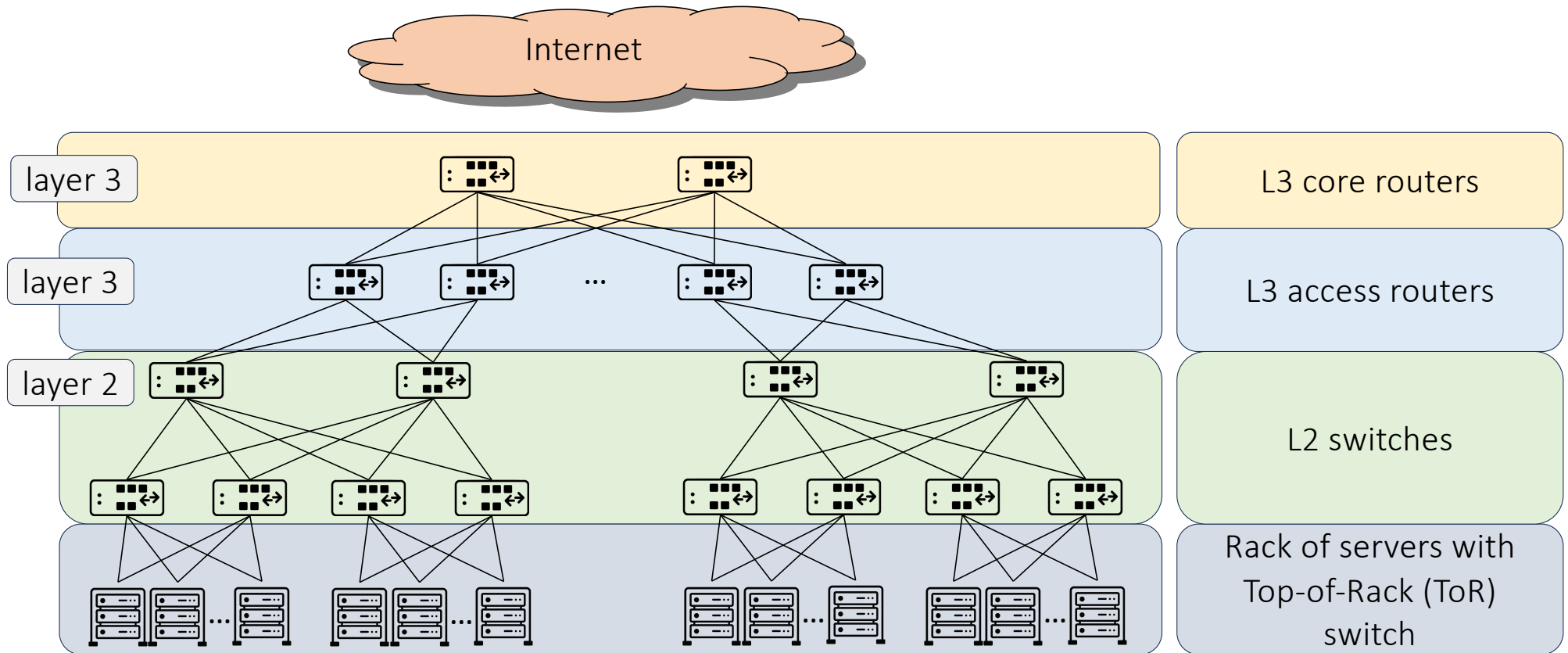


# Conventional Datacenter Network

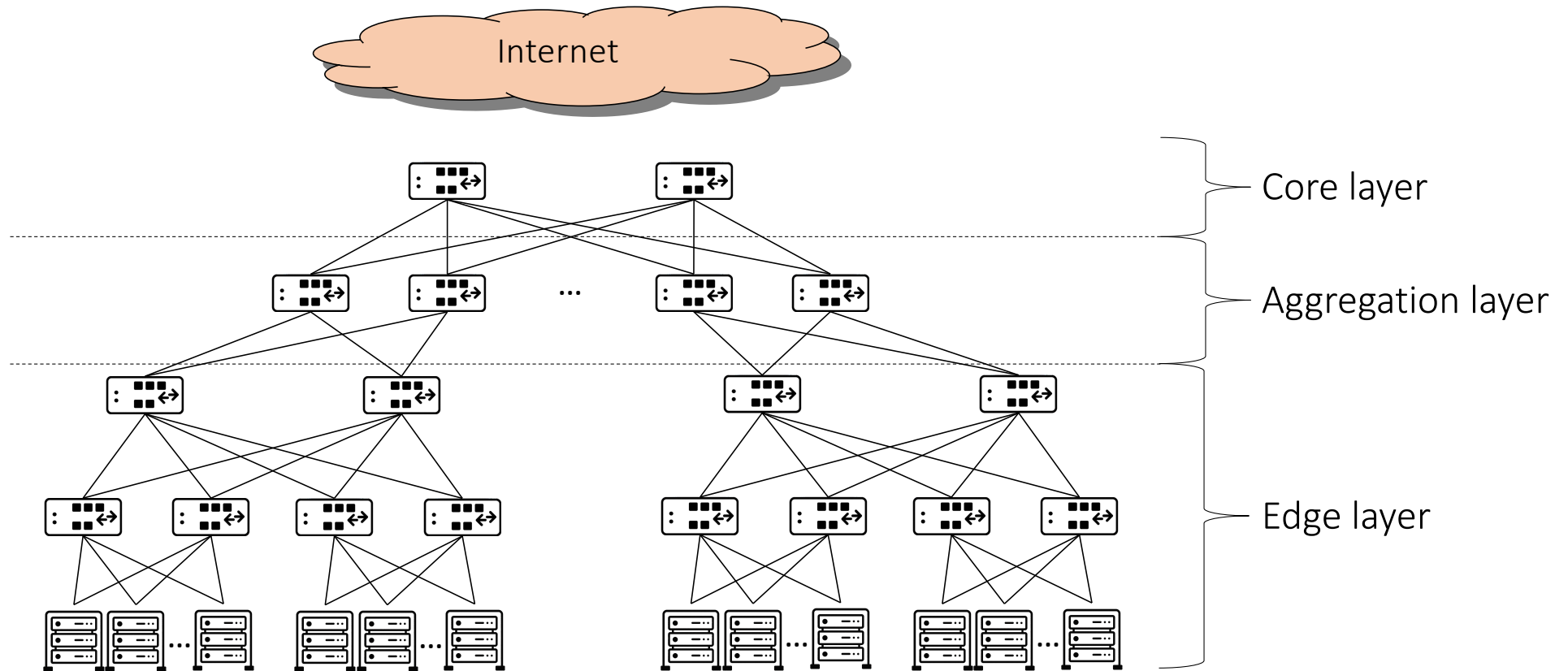




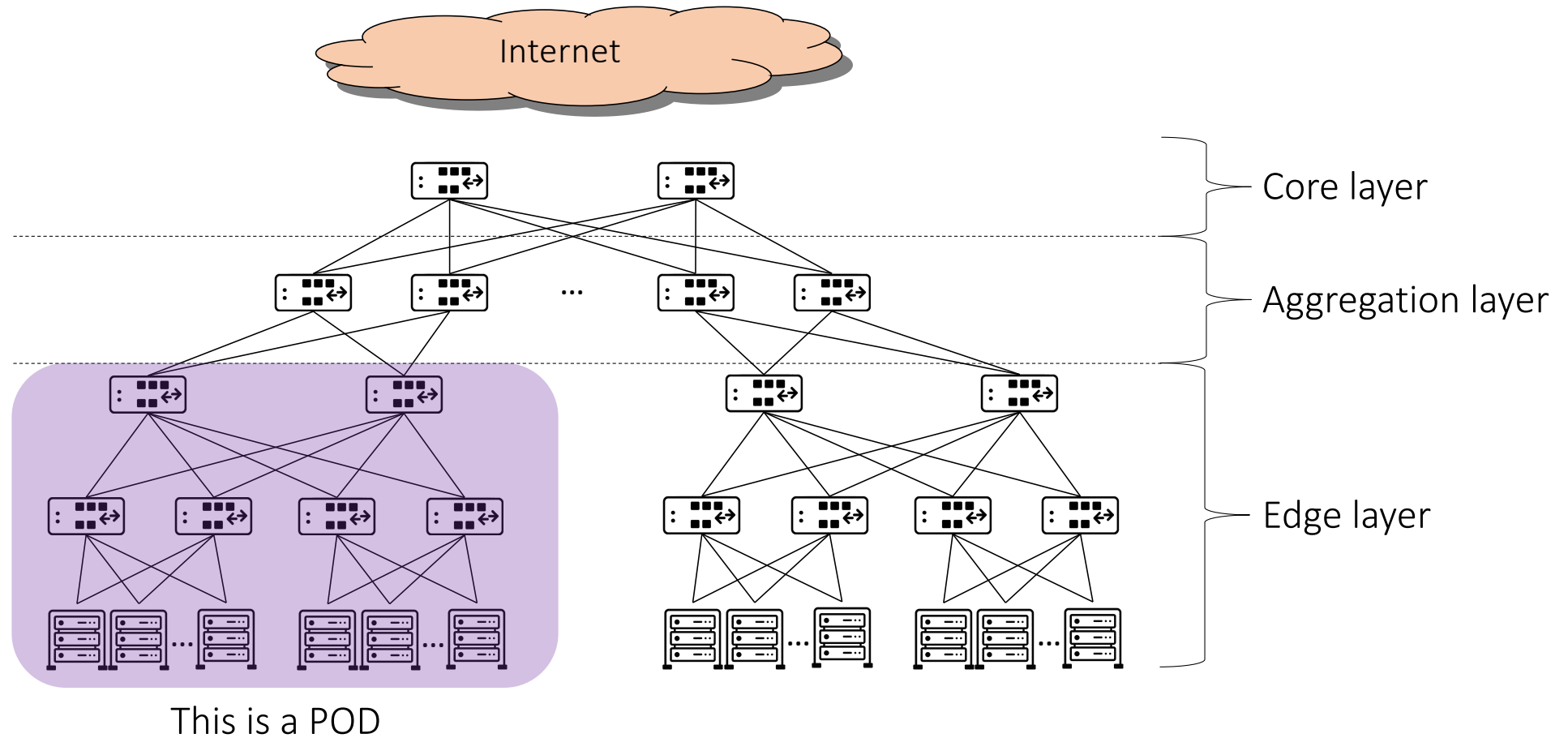
# Conventional Datacenter Network



# Conventional Datacenter Network



# Conventional Datacenter Network



# Terminology

- Diameter: max number of hops between any two node

# Terminology

- Diameter: max number of hops between any two node
- Bisection bandwidth: if the network is divided into two *equal* partitions, the bisection bandwidth of a network topology is the bandwidth available between the two partitions

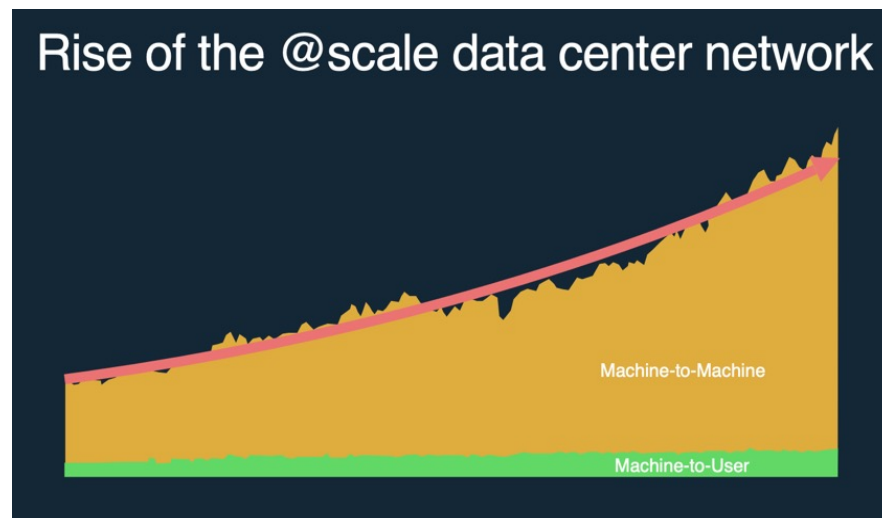
# Terminology

- Diameter: max number of hops between any two node
- Bisection bandwidth: if the network is divided into two *equal* partitions, the bisection bandwidth of a network topology is the bandwidth available between the two partitions
- Full bisection bandwidth: one half of the nodes can communicate simultaneously with other half of the nodes

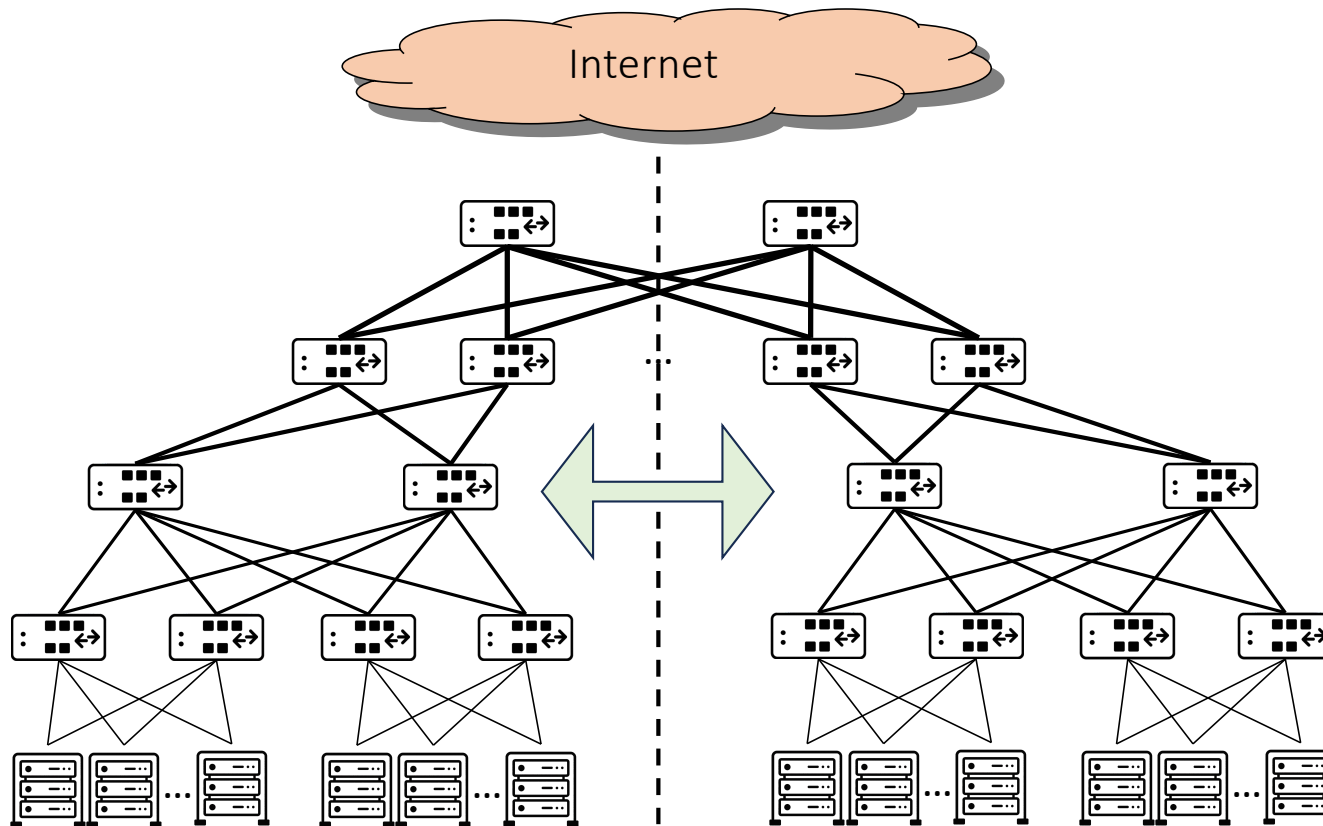
# The quest for high-bandwidth

Ideally each server shall be able to talk to any other server at its full access link rate (lots of east-west traffic)

Remember this? 😊



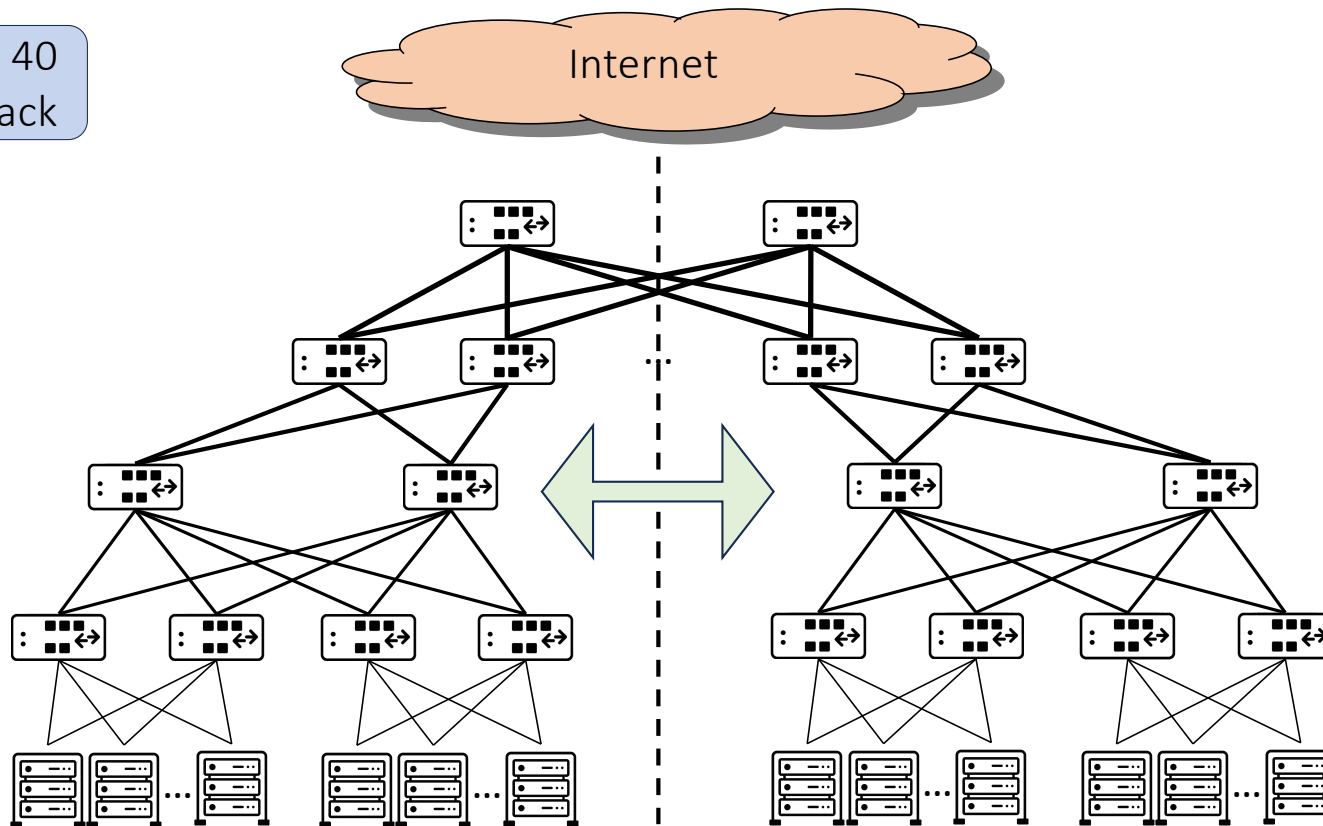
# The quest for full-bisection bandwidth





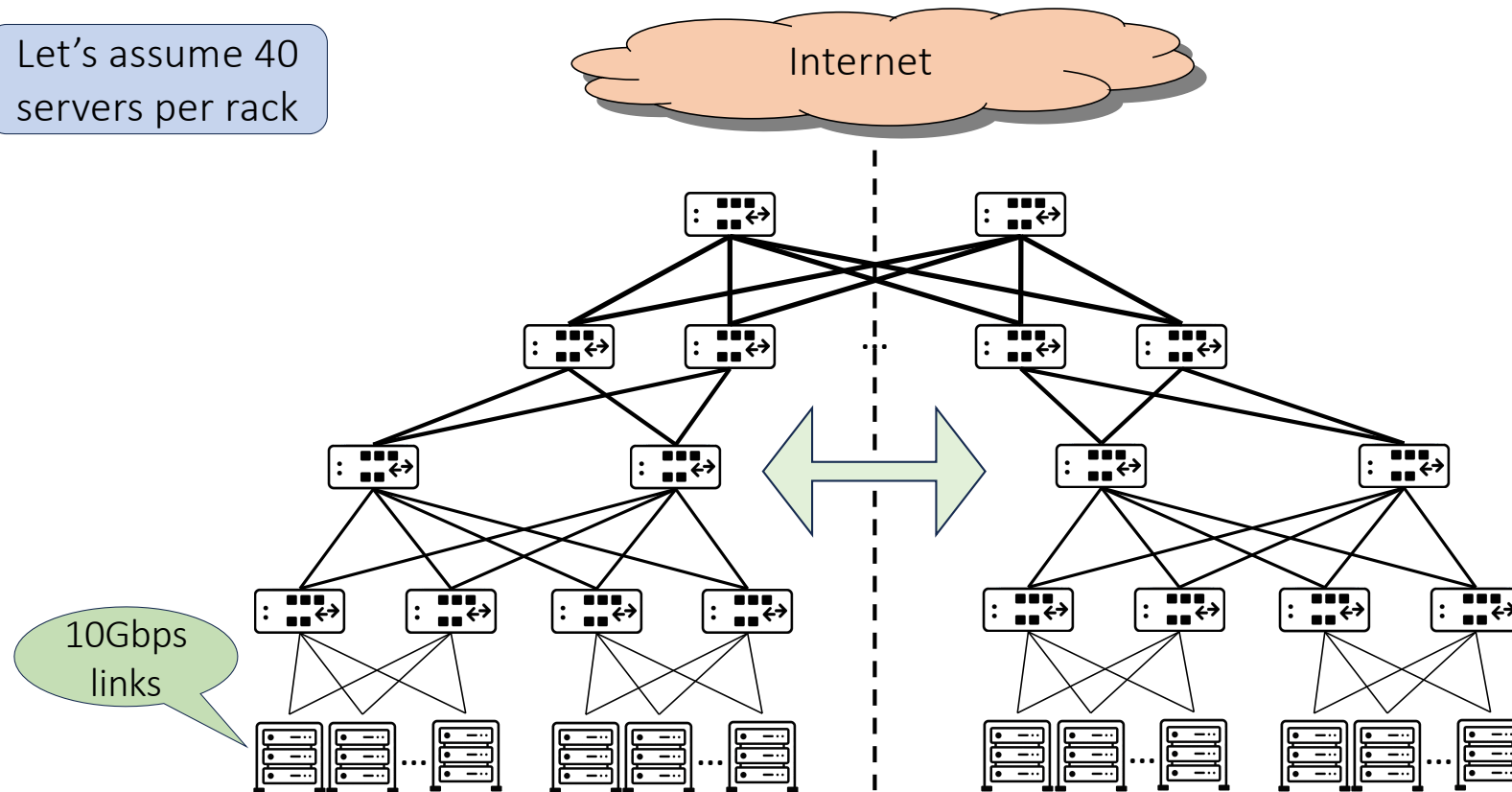
# The quest for full-bisection bandwidth

Let's assume 40  
servers per rack



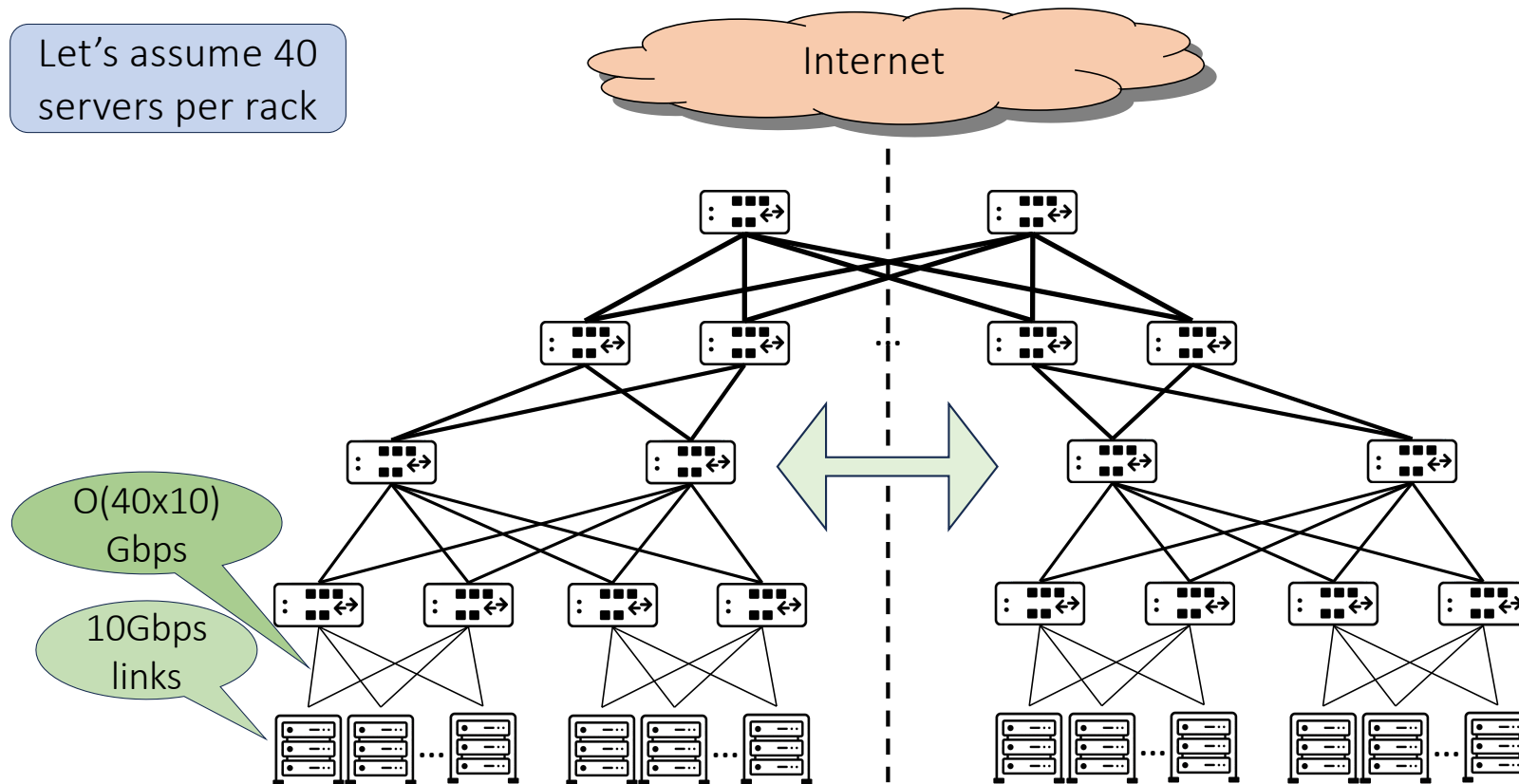
# The quest for full-bisection bandwidth

Let's assume 40  
servers per rack

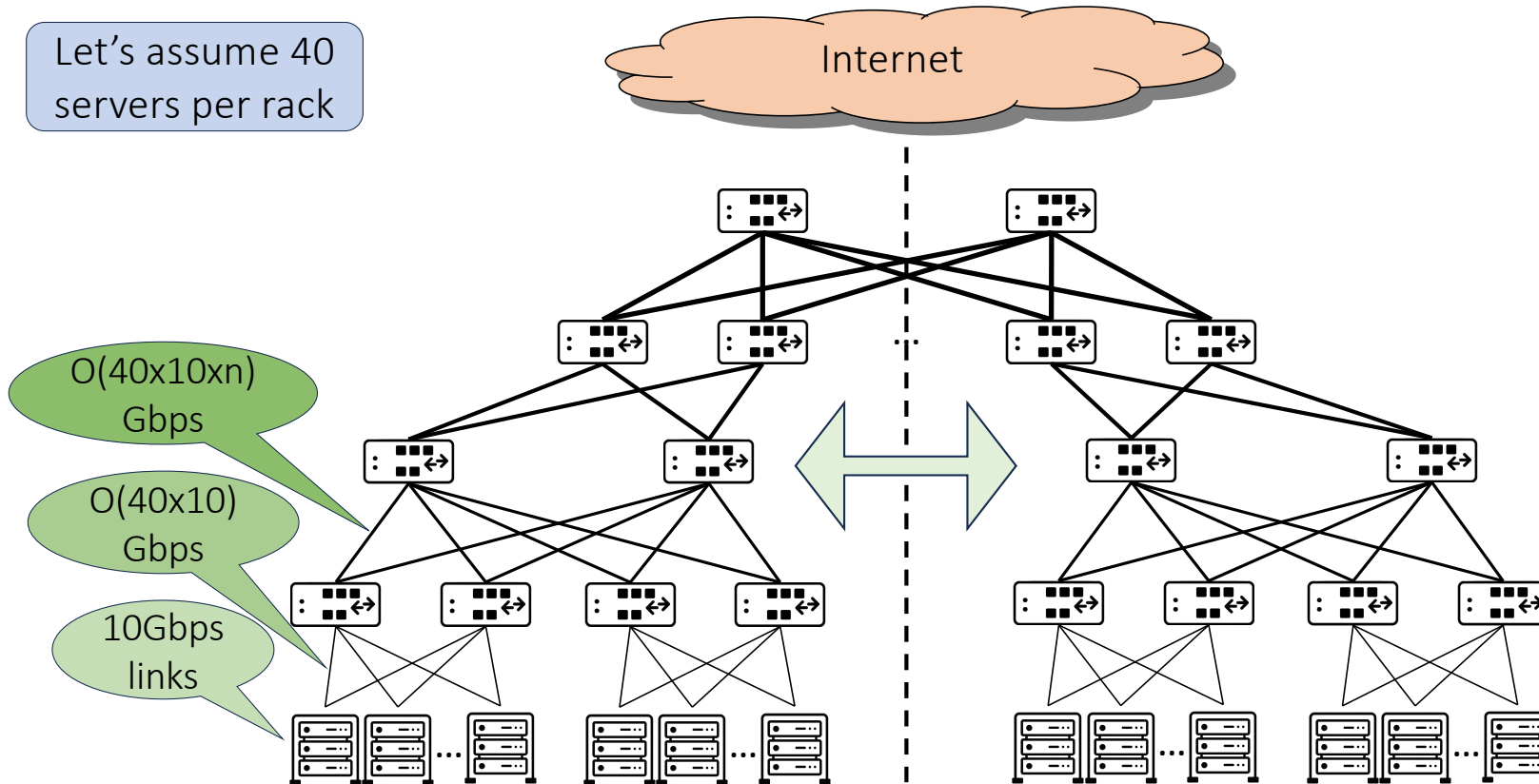


# The quest for full-bisection bandwidth

Let's assume 40  
servers per rack



# The quest for full-bisection bandwidth



# Full bisection bandwidth?

- Scaling up a traditional tree topology is very expensive!
- Solutions?

# Full bisection bandwidth?

- Scaling up a traditional tree topology is very expensive!
- Solutions
  - Over-subscribe (provision less than full bandwidth)

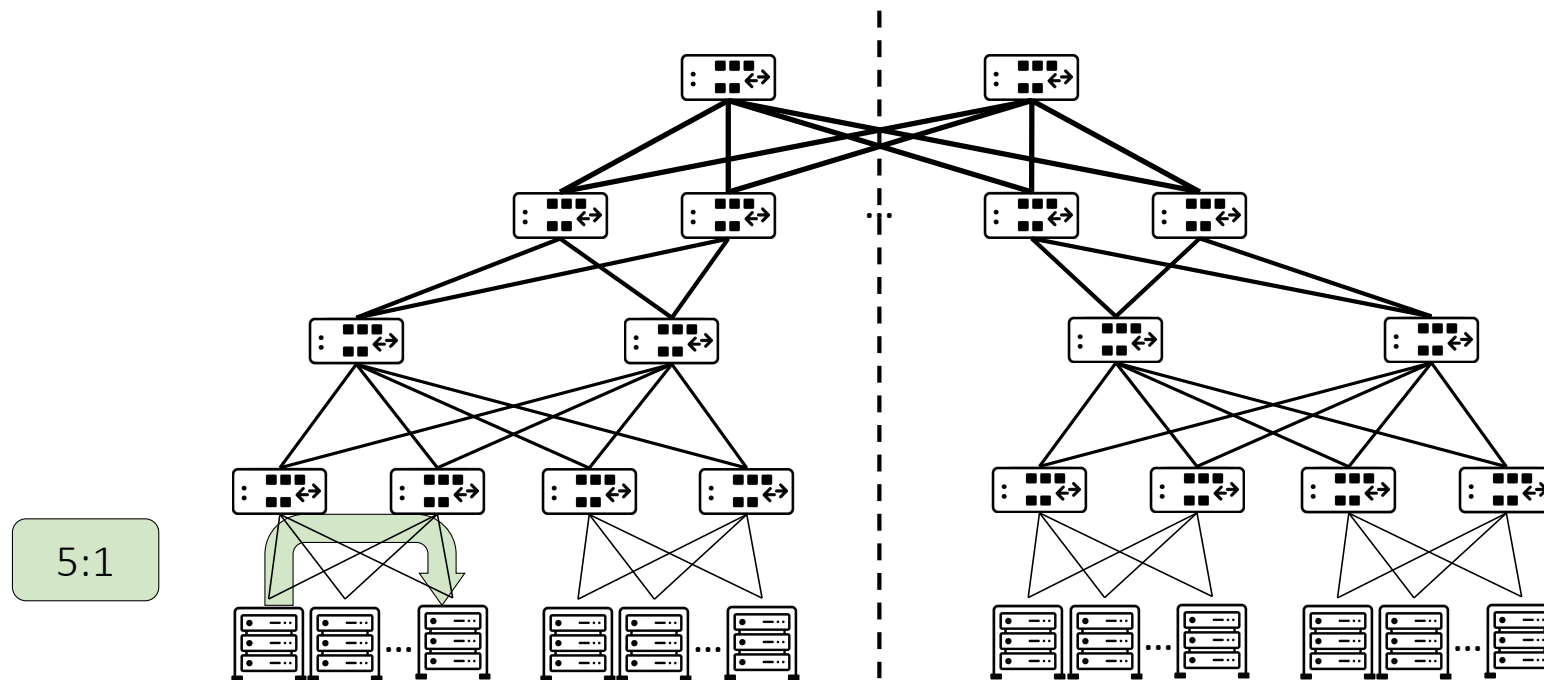
# Full bisection bandwidth?

- Scaling up a traditional tree topology is very expensive!
- Solutions
  - Over-subscribe (provision less than full bandwidth)

over-subscription is the *ratio* of the worst-case achievable aggregate bandwidth among the end hosts to the total bisection bandwidth of a particular communication topology. An oversubscription of 1:1 indicates that all hosts may potentially communicate with arbitrary other hosts at the full bandwidth of their network interface

# The quest for full-bisection bandwidth

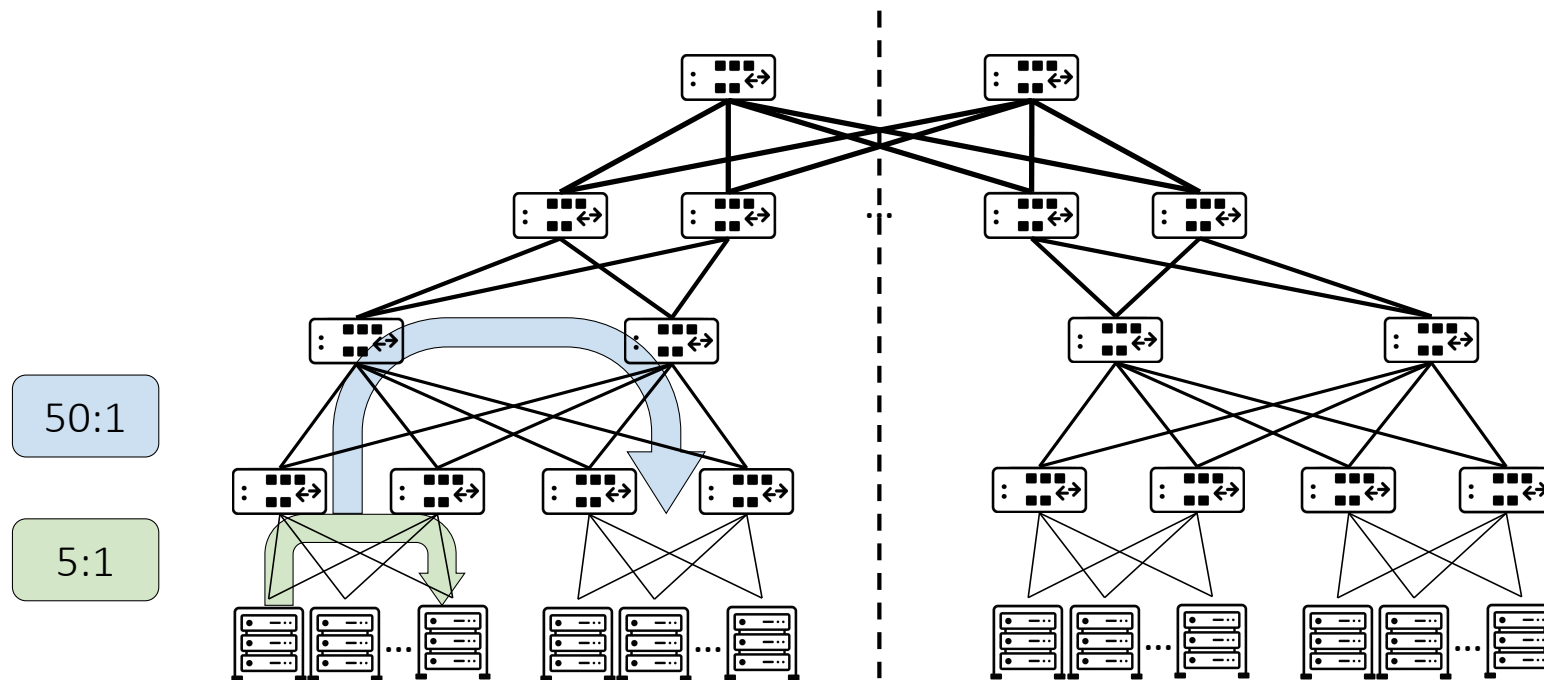
This means that only 20% of available host bandwidth is available for some communication patterns





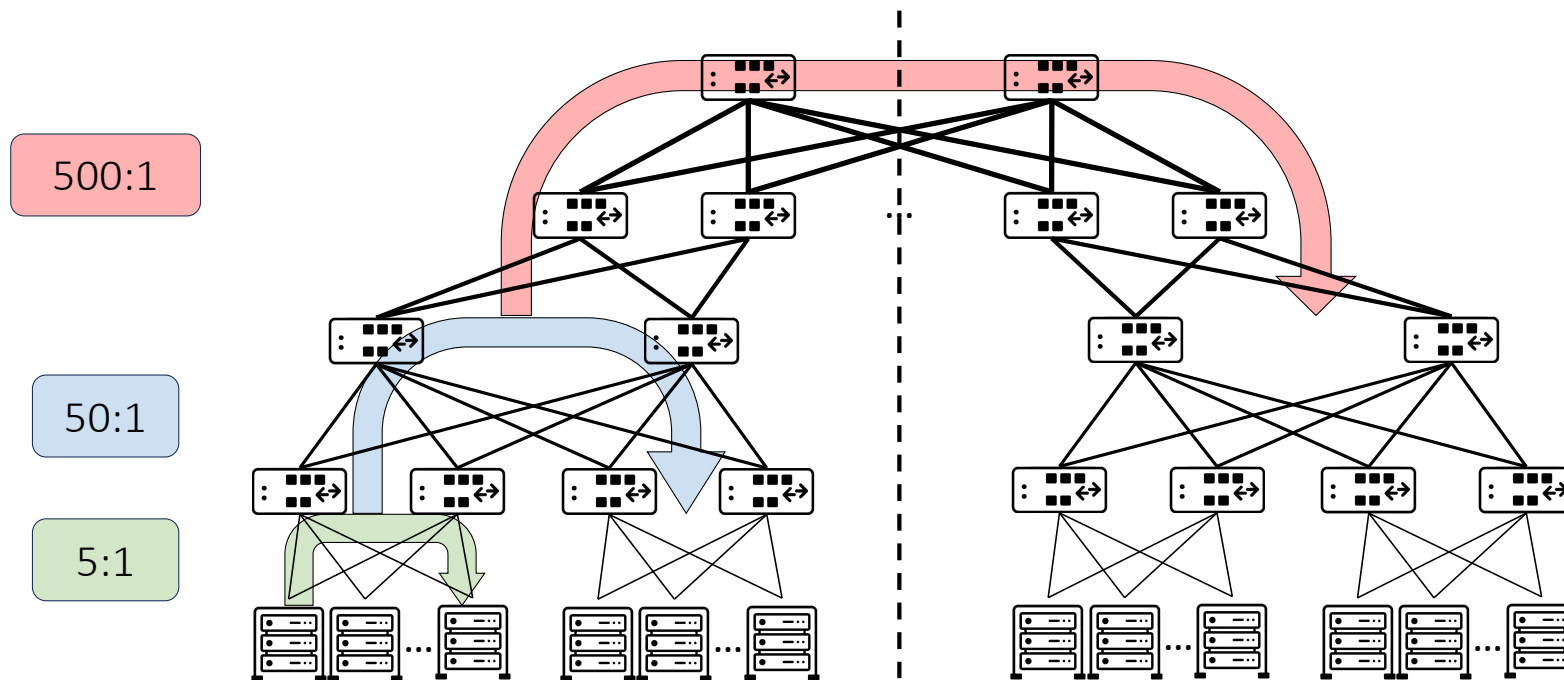
# The quest for full-bisection bandwidth

This means that only 2% of available host bandwidth is available for some communication patterns



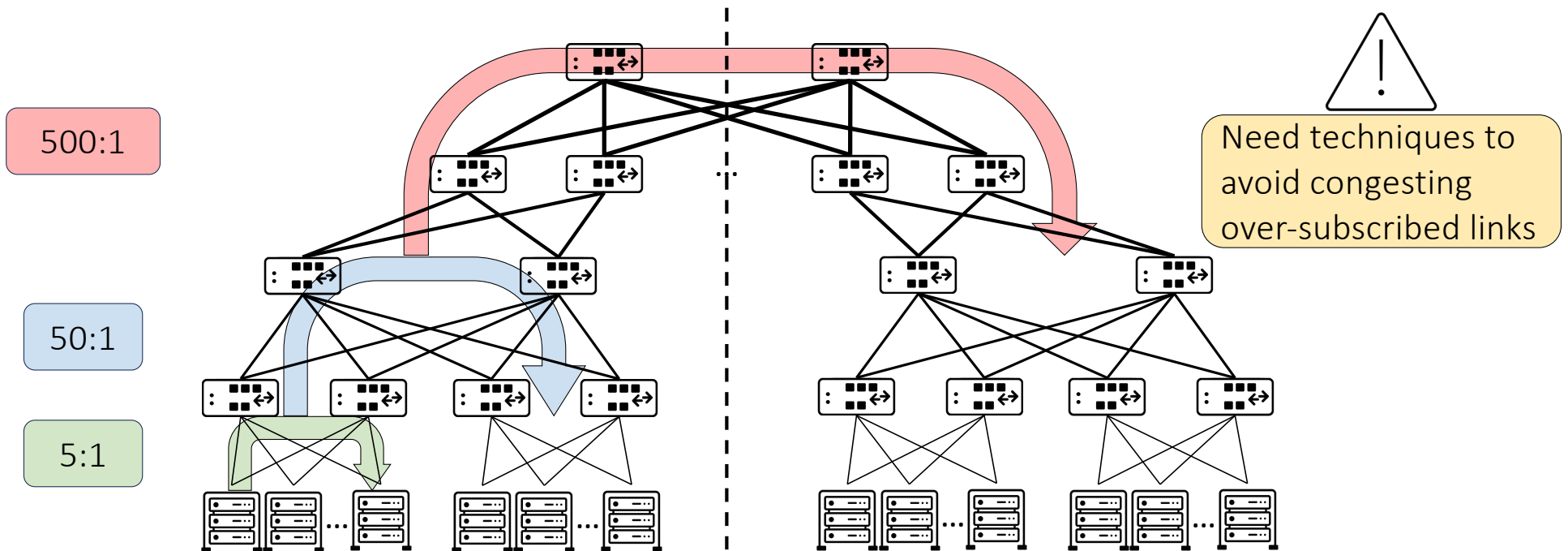
# The quest for full-bisection bandwidth

This means that only 0.5% of available host bandwidth is available for some communication patterns



# The quest for full-bisection bandwidth

This means that only 0.5% of available host bandwidth is available for some communication patterns



# Full bisection bandwidth?

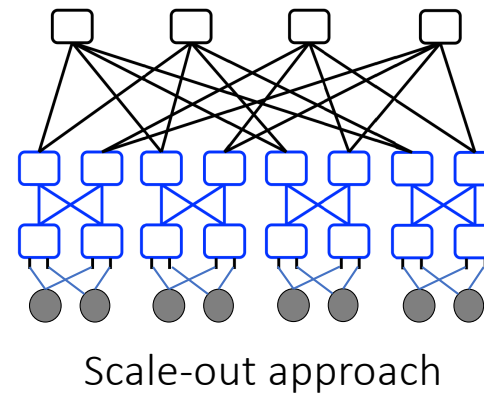
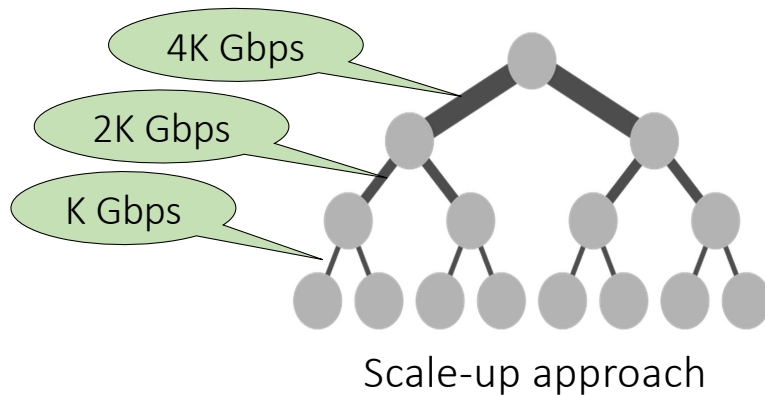
- Scaling up a traditional tree topology is very expensive!
- Solutions
  - Over-subscribe (provision less than full bandwidth)
  - ?

# Full bisection bandwidth?

- Scaling up a traditional tree topology is very expensive!
- Solutions
  - Over-subscribe (provision less than full bandwidth)
  - Better topologies

# Full bisection bandwidth?

- Scaling up a traditional tree topology is very expensive!
- Solutions
  - Over-subscribe (provision less than full bandwidth)
  - Better topologies



# Some Other Topologies

## A Scalable, Commodity Data Center Network Architecture

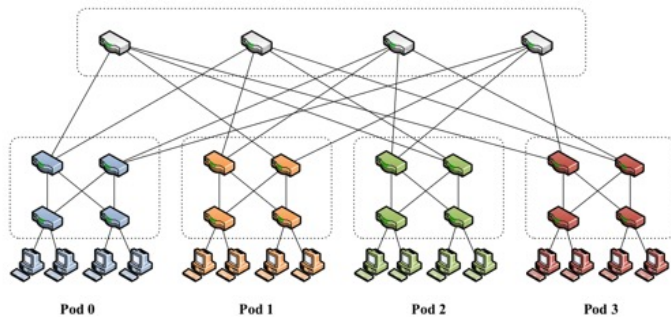
Mohammad Al-Fares  
mallares@cs.ucsd.edu

Alexander Loukissas  
aloukiss@cs.ucsd.edu

Amin Vahdat  
vahdat@cs.ucsd.edu

Department of Computer Science and Engineering  
University of California, San Diego  
La Jolla, CA 92093-0404

Fat Tree  
[SIGCOMM 2008]

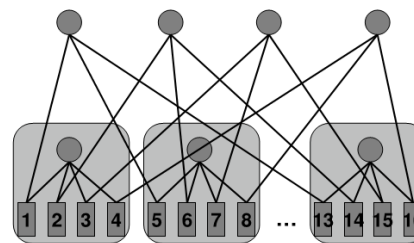


## BCube: A High Performance, Server-centric Network Architecture for Modular Data Centers

Chuanxiong Guo<sup>1</sup>, Guohan Lu<sup>1</sup>, Dan Li<sup>1</sup>, Haitao Wu<sup>1</sup>, Xuan Zhang<sup>1,2</sup>, Yunfeng Shi<sup>1,3</sup>,  
Chen Tian<sup>1,4</sup>, Yongguang Zhang<sup>1</sup>, Songwu Lu<sup>1,3</sup>

<sup>1</sup>: Microsoft Research Asia, <sup>2</sup>: Tsinghua, <sup>3</sup>: PKU, <sup>4</sup>: HUST, <sup>5</sup>: UCLA  
{chguo, lguohan, daniel, hwu}@microsoft.com, xuan-zhang05@mails.tsinghua.edu.cn,  
shiyunfeng@pku.edu.cn, tianchen@mail.hust.edu.cn, ygz@microsoft.com,  
slu@cs.ucla.edu

Bcube  
[SIGCOMM 2009]



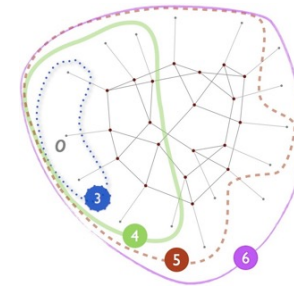
## Jellyfish: Networking Data Centers Randomly

Ankit Singla<sup>†\*</sup>, Chi-Yao Hong<sup>†\*</sup>, Lucian Popa<sup>‡</sup>, P. Brighten Godfrey<sup>†</sup>

<sup>†</sup> University of Illinois at Urbana-Champaign

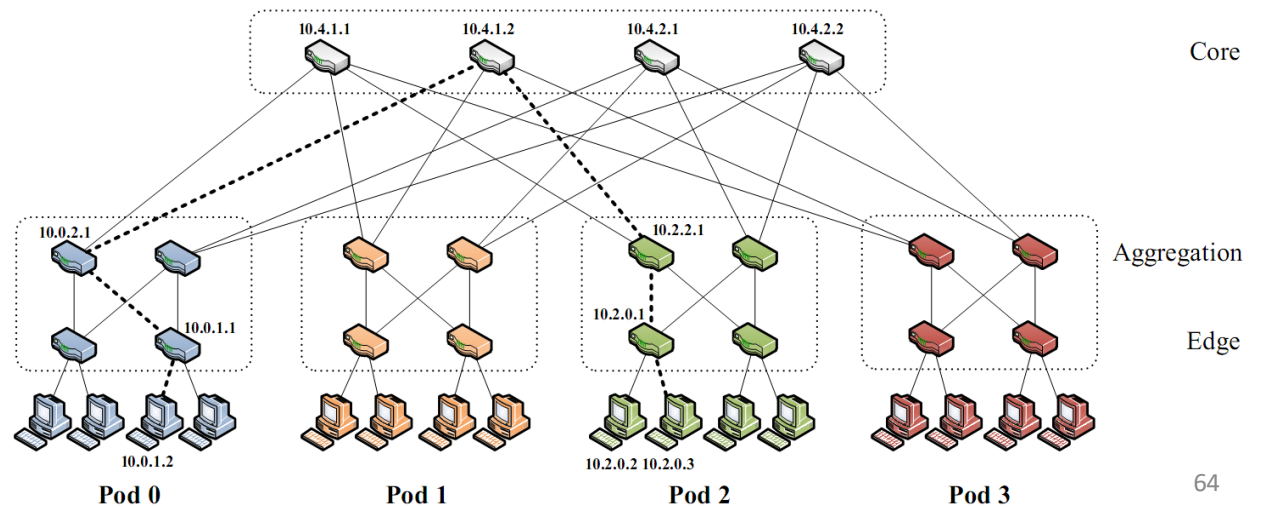
<sup>‡</sup> HP Labs

Jellyfish  
[NSDI 2012]



# Fat tree: a special type of Clos Networks

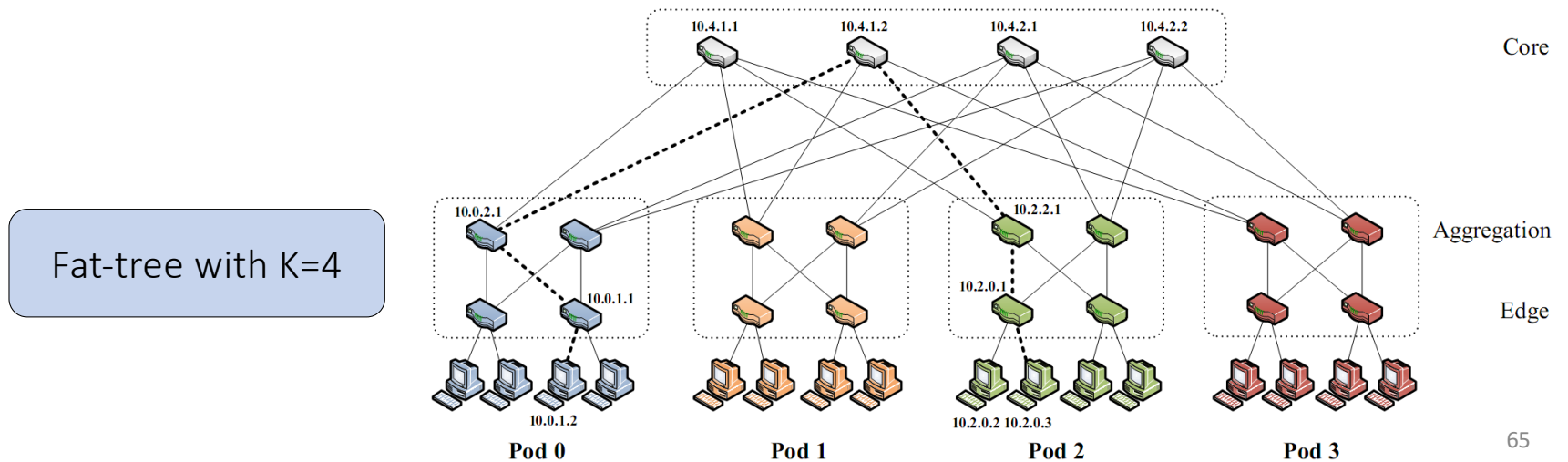
A Clos network is a type of non-blocking, multistage switching architecture that reduces the number of ports required in an interconnected fabric



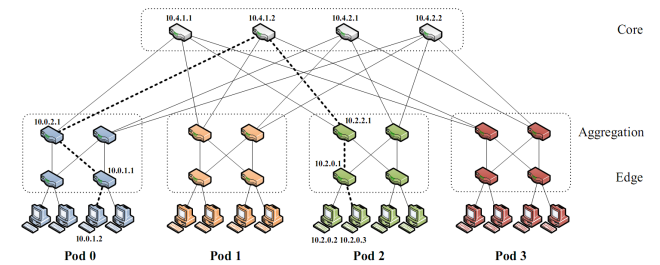


# Fat tree: a special type of Clos Networks

- K-ary fat tree: three-layer topology (edge, aggregation and core)
- Each pod consists of  $(k/2)^2$  servers & 2 layers of  $k/2$  k-port switches
- Each edge switch connects to  $k/2$  servers &  $k/2$  aggr. switches
- Each aggr. switch connects to  $k/2$  edge &  $k/2$  core switches
- $(k/2)^2$  core switches: each connects to  $k$  pods

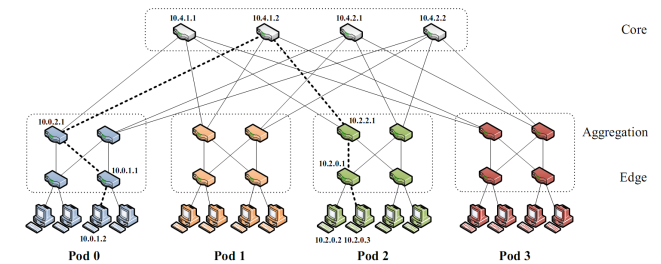


# Why Fat tree?



- Can be built using cheap devices with uniform capacity
  - Each port supports same speed as end host
  - All devices can transmit at line speed if packets are distributed uniformly across available paths

# Why Fat tree?



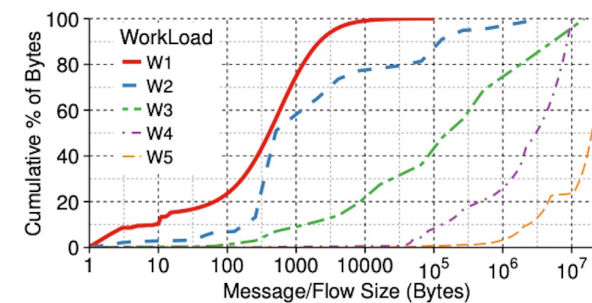
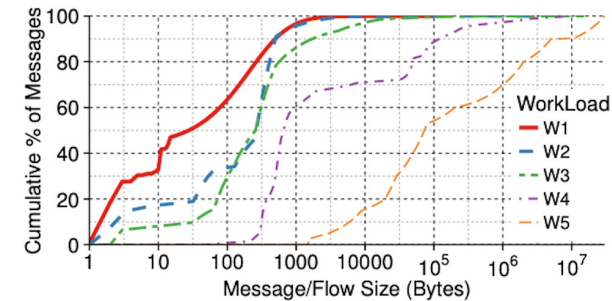
- Can be built using cheap devices with uniform capacity
  - Each port supports same speed as end host
  - All devices can transmit at line speed if packets are distributed uniformly across available paths
- All other system components must be able to exploit the available capacity:
  - Routing must use all paths
  - Transport protocol must fill all pipes (fast)

# Exploiting available capacity is hard!

- Lots of flows running simultaneously
- Traffic is neither rack-local nor all-to-all: locality depends upon the services [Facebook/Meta 2015]
- Traffic is bursty [Facebook/Meta 2022]
- Flows have different characteristics

# Exploiting available capacity is hard!

- Lots of flows running simultaneously
- Traffic is neither rack-local nor all-to-all: locality depends upon the services [Facebook/Meta 2015]
- Traffic is bursty [Facebook/Meta 2022]
- Flows have different characteristics



W1 Accesses to a collection of memcached servers at Facebook, as approximated by the statistical model of the ETC workload in Section 5 and Table 5 of [5].

W2 Search application at Google [29].

W3 Aggregated workload from all applications running in a Google datacenter [29].

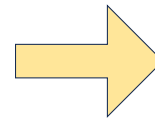
W4 Hadoop cluster at Facebook [27].

W5 Web search workload used for DCTCP [2].

# Datacenter workloads



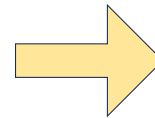
Short messages  
*(e.g., query, coordination)*



low latency



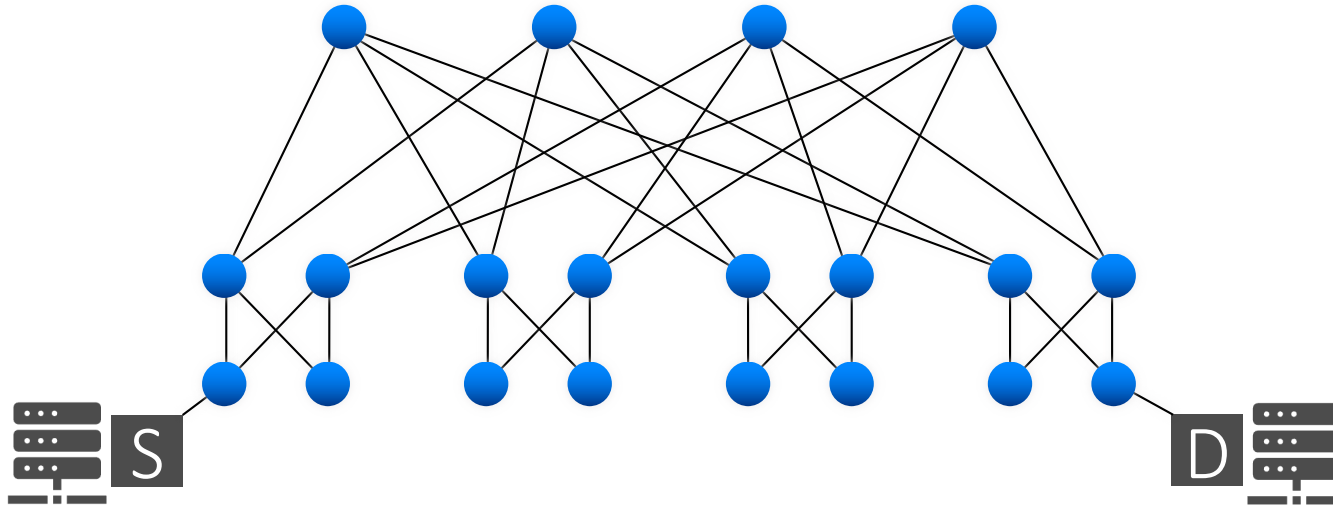
Large flows  
*(e.g., data update, backup)*



high throughput

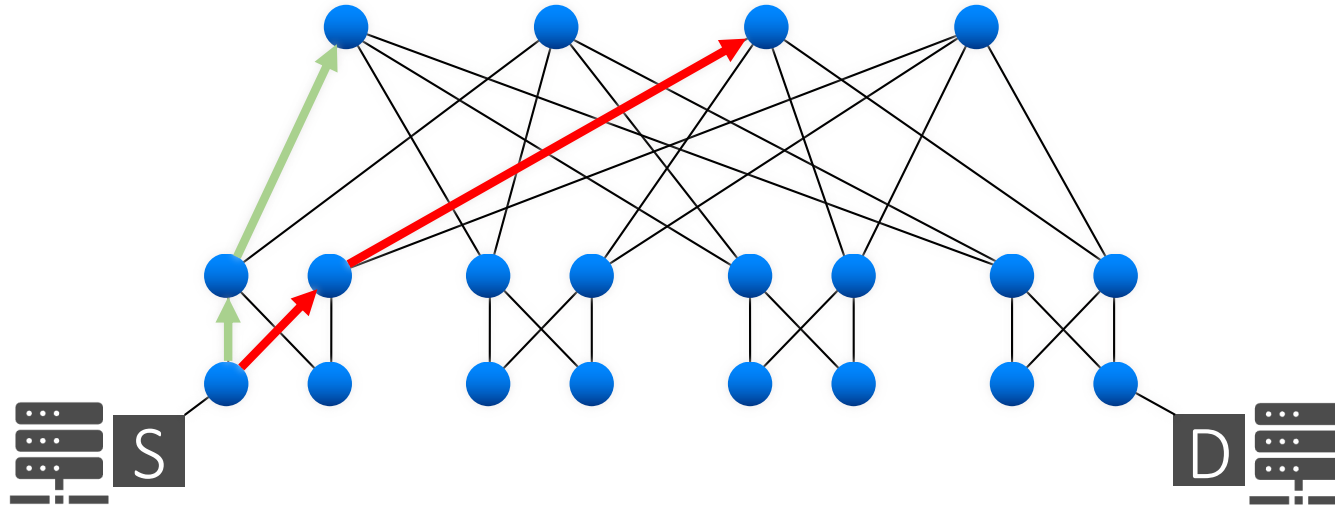
# This makes life hard for routing!

- Let's assume many flows from S to D
- Many paths are possible



# This makes life hard for routing!

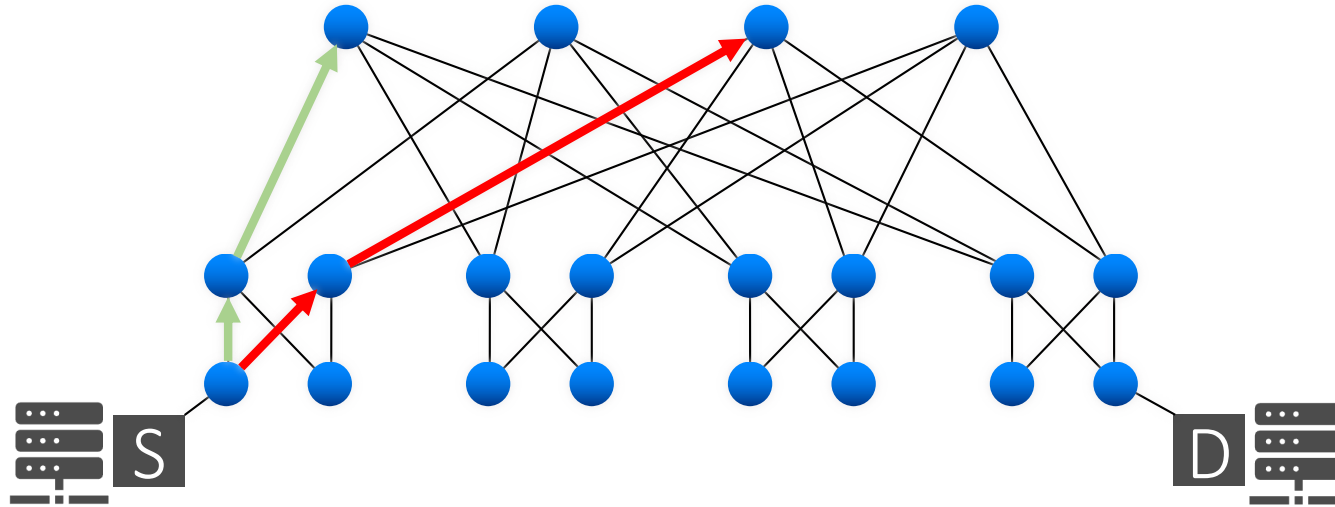
- Let's assume many flows from S to D
- Many paths are possible
- You can randomly pick a path





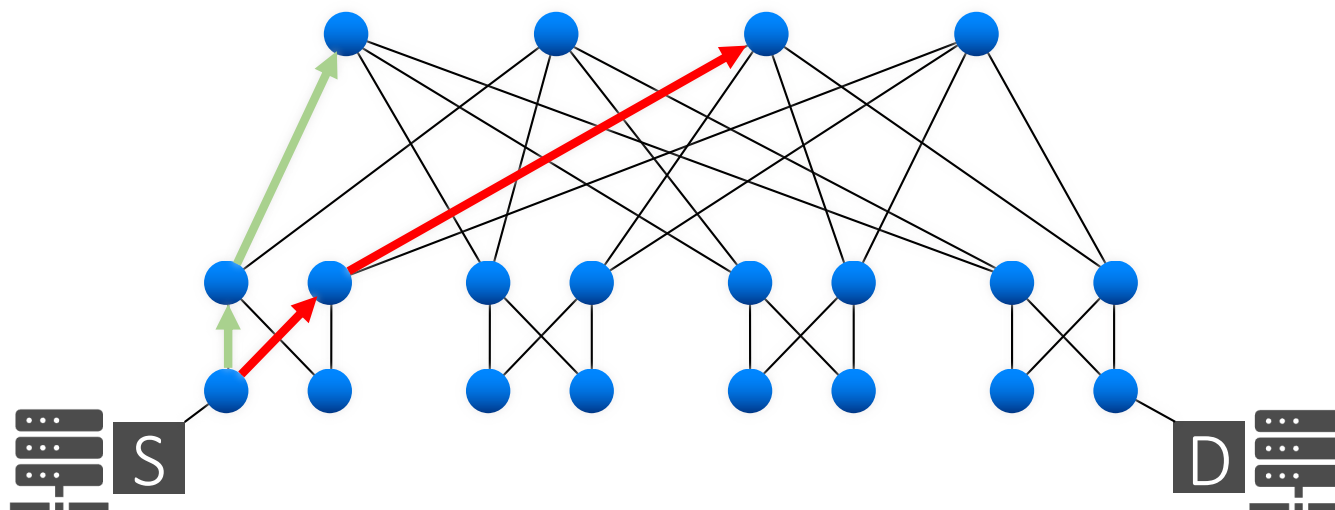
# This makes life hard for routing!

- Problems?



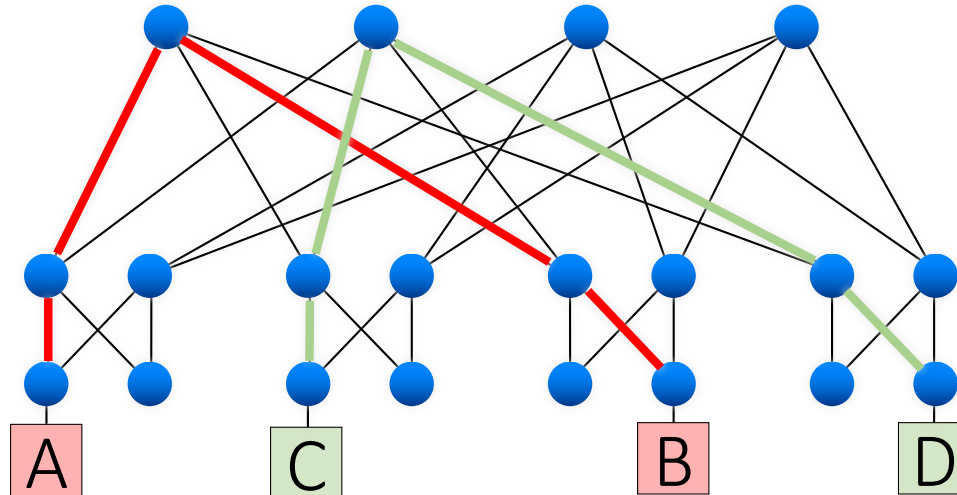
# This makes life hard for routing!

- Agnostic to available resources
- Long-lasting collisions with elephant flows



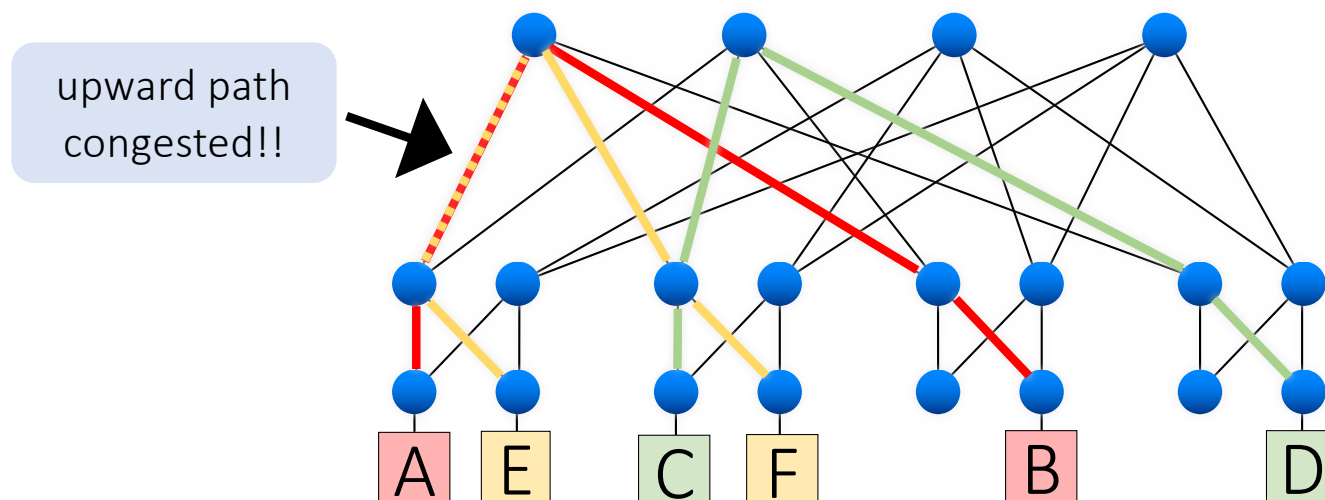
# This makes life hard for routing!

- This would be ideal 😊



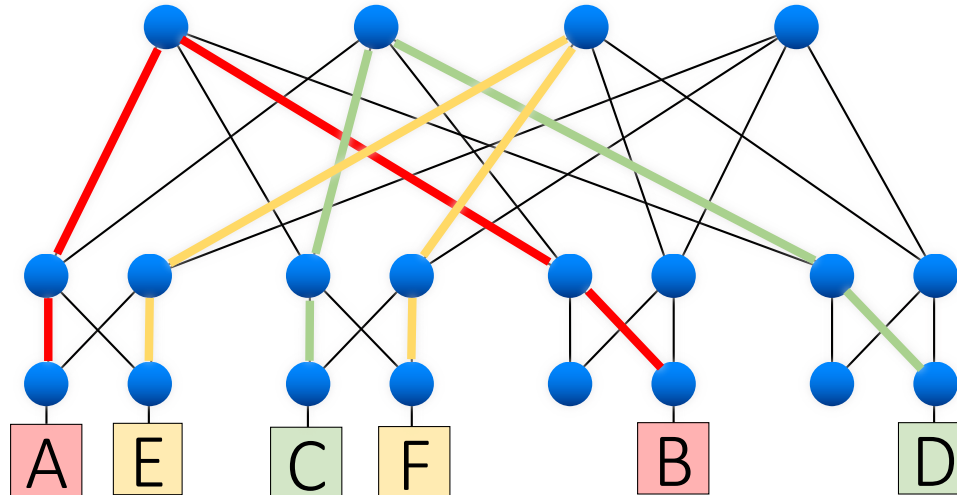
# This makes life hard for routing!

- This would be ideal 😊
- Then, suddenly a short flow arrives and...this is less ideal 😞



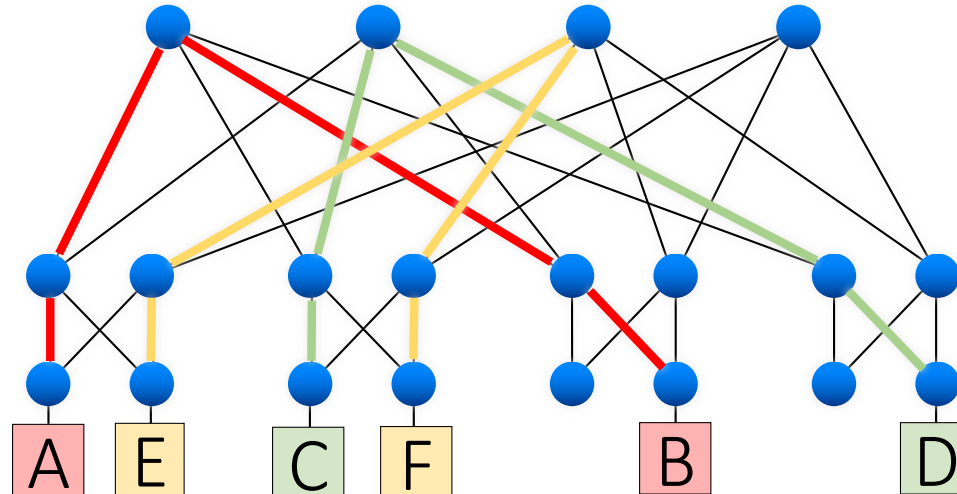
# This makes life hard for routing!

- This would be ideal 😊
- Then, suddenly a short flow arrives and...this is less ideal 😞
- ...and the network would have enough capacity! 😞



# This makes life hard for routing!

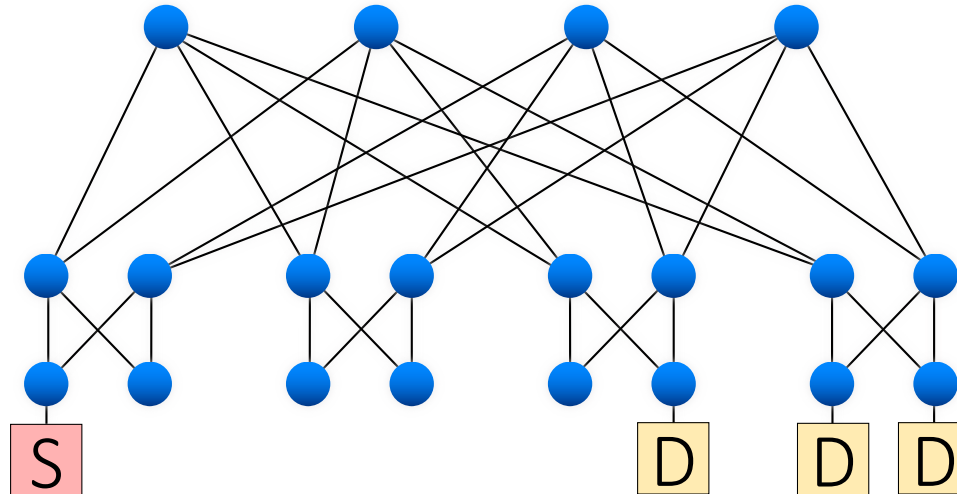
- This would be ideal 😊
- Then, suddenly a short flow arrives and...this is less ideal 😞
- ...and the network would have enough capacity! 😞



Food for thoughts for researchers. We will see more about this

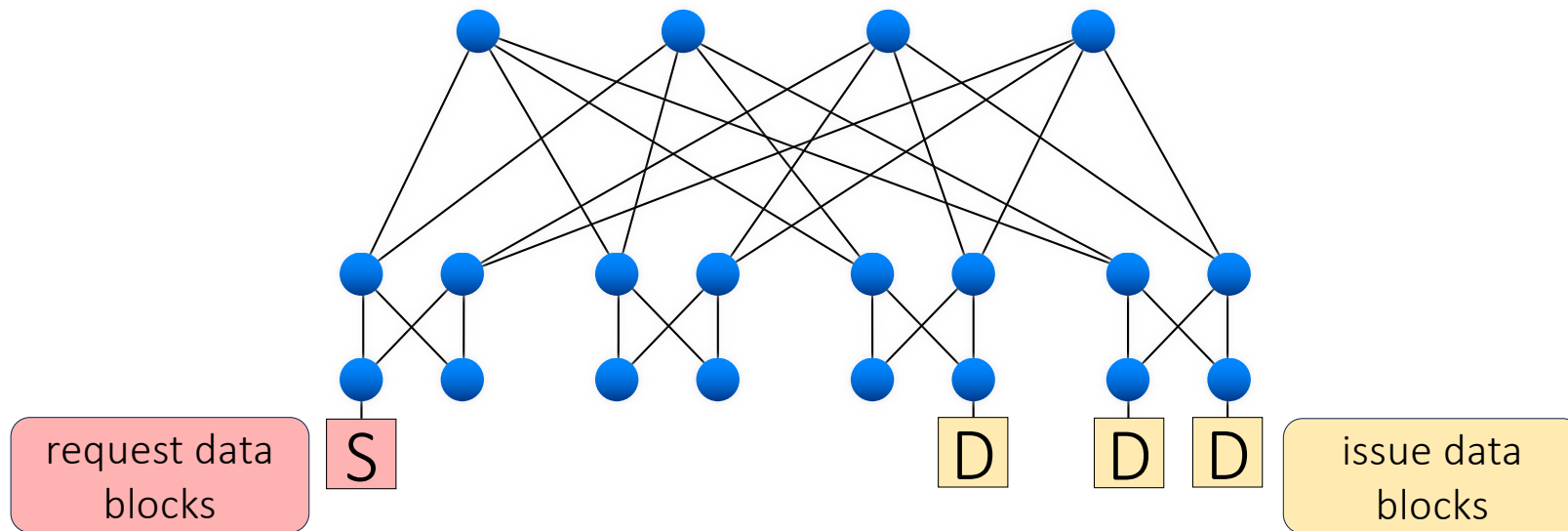
# Unfortunately, it is not only about routing

- In a cluster file system, a client application requests a data block striped across several storage servers, issuing the next data block request only when all servers have responded with their portion



# Unfortunately, it is not only about routing

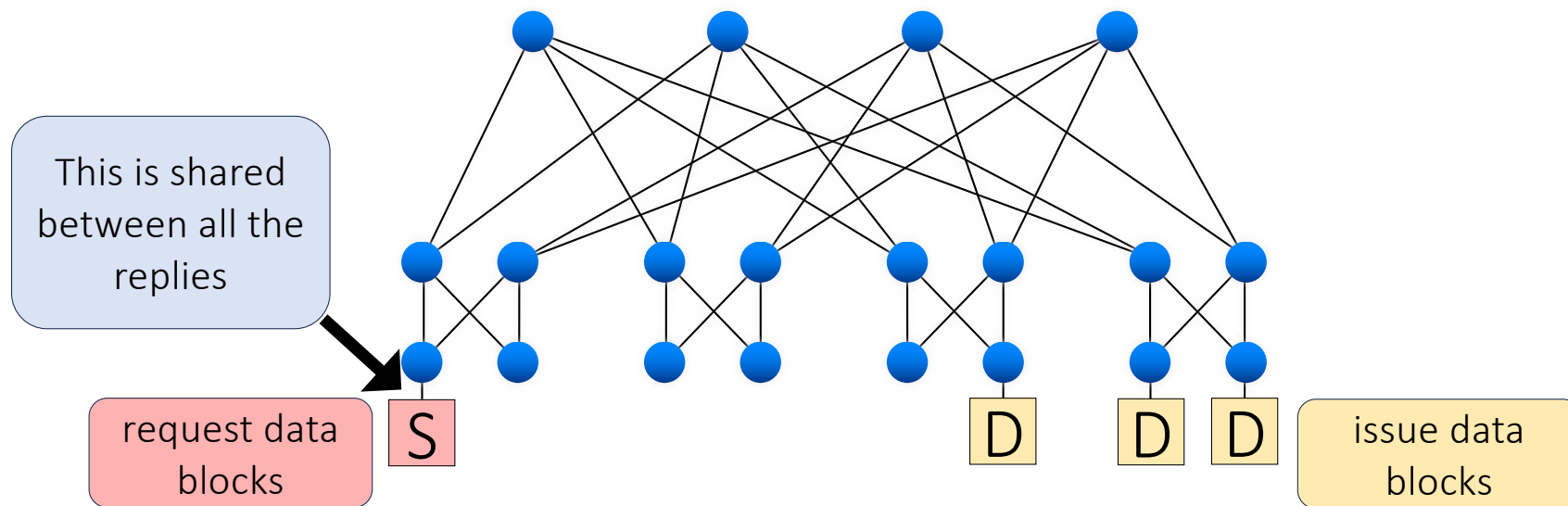
- In a cluster file system, a client application requests a data block striped across several storage servers, issuing the next data block request only when all servers have responded with their portion



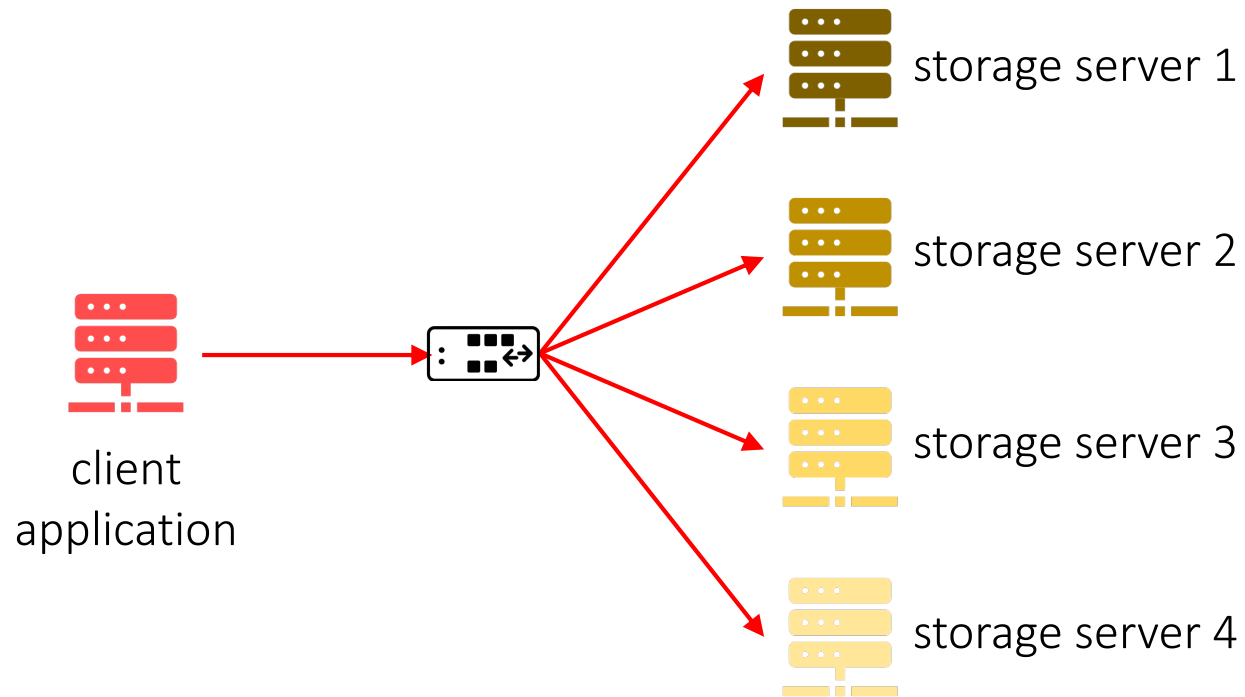


# Unfortunately, it is not only about routing

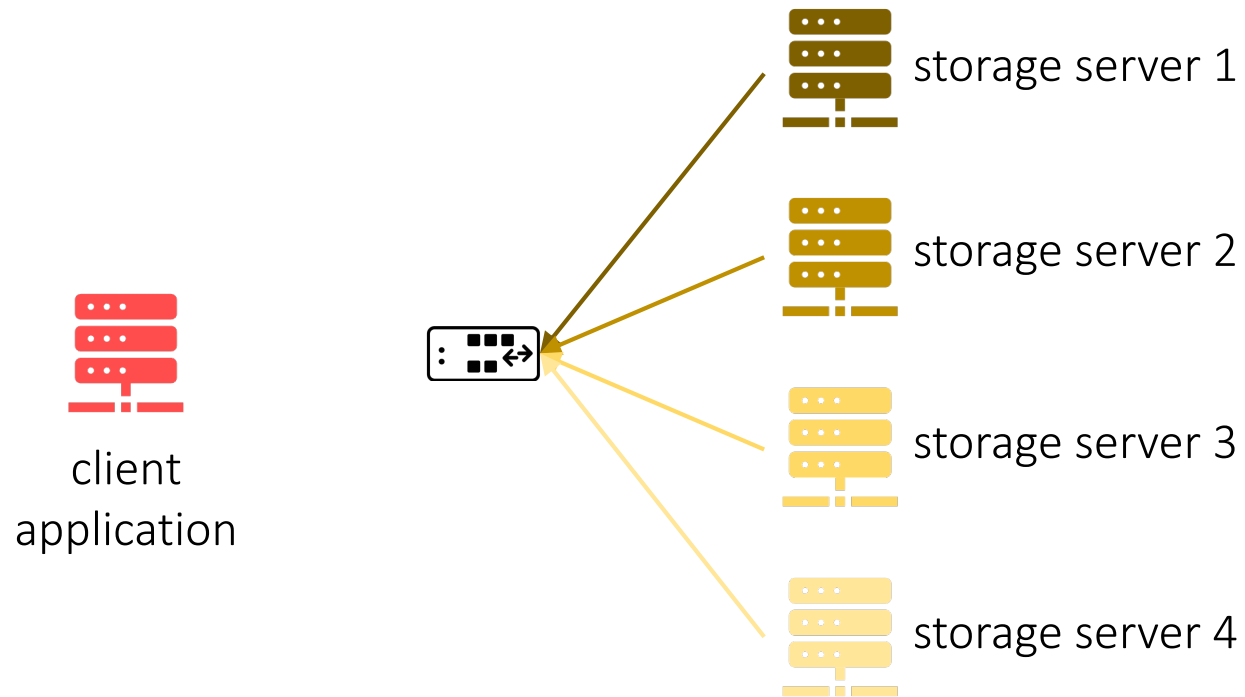
- In a cluster file system, a client application requests a data block striped across several storage servers, issuing the next data block request only when all servers have responded with their portion



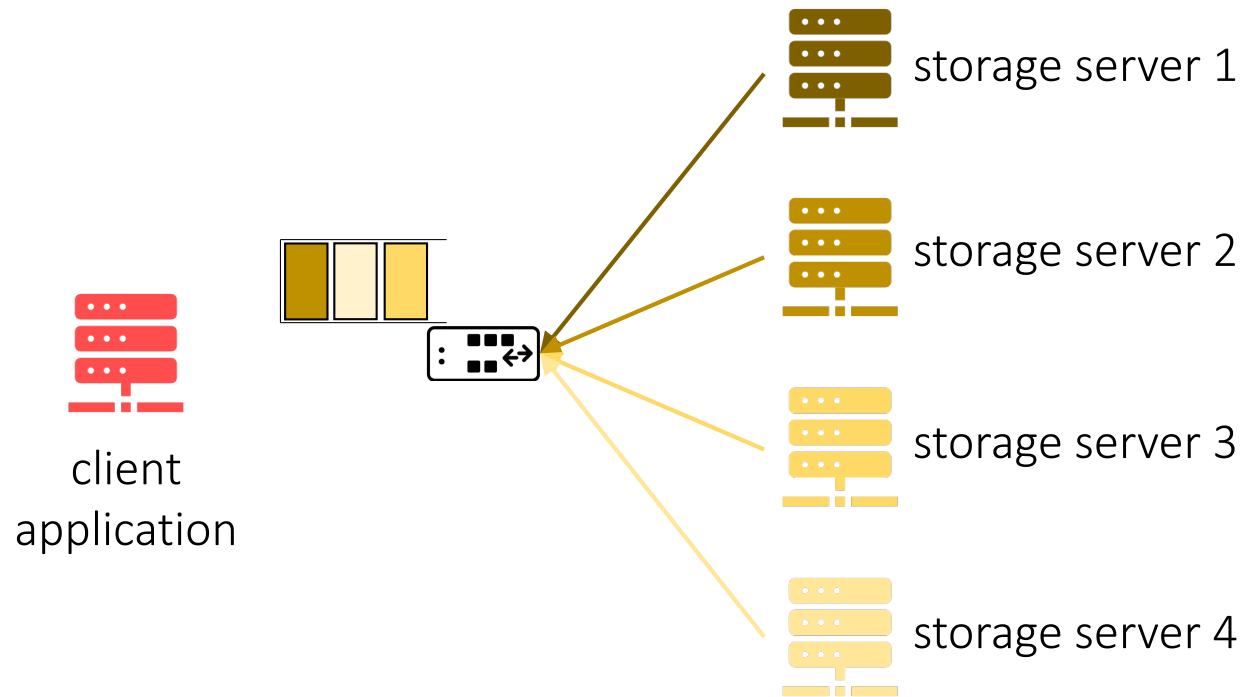
# This is what it is called TCP Incast



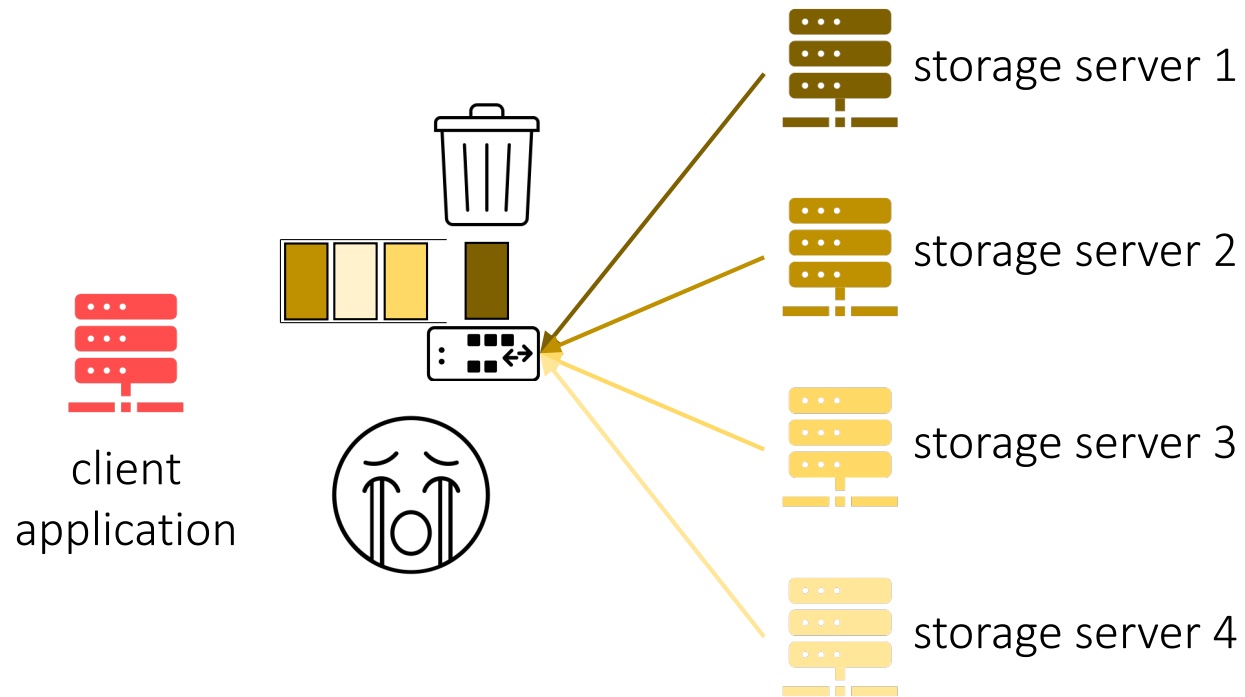
# This is what it is called TCP Incast



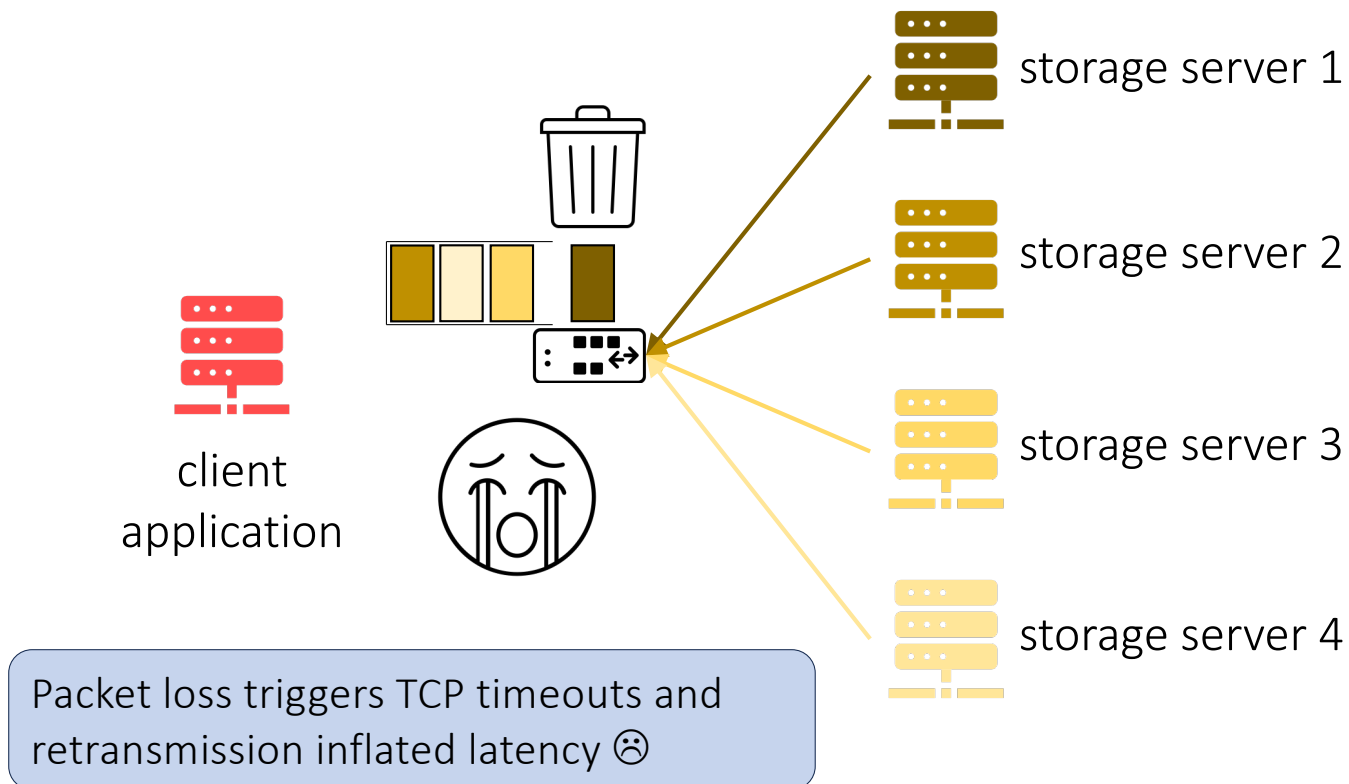
# This is what it is called TCP Incast



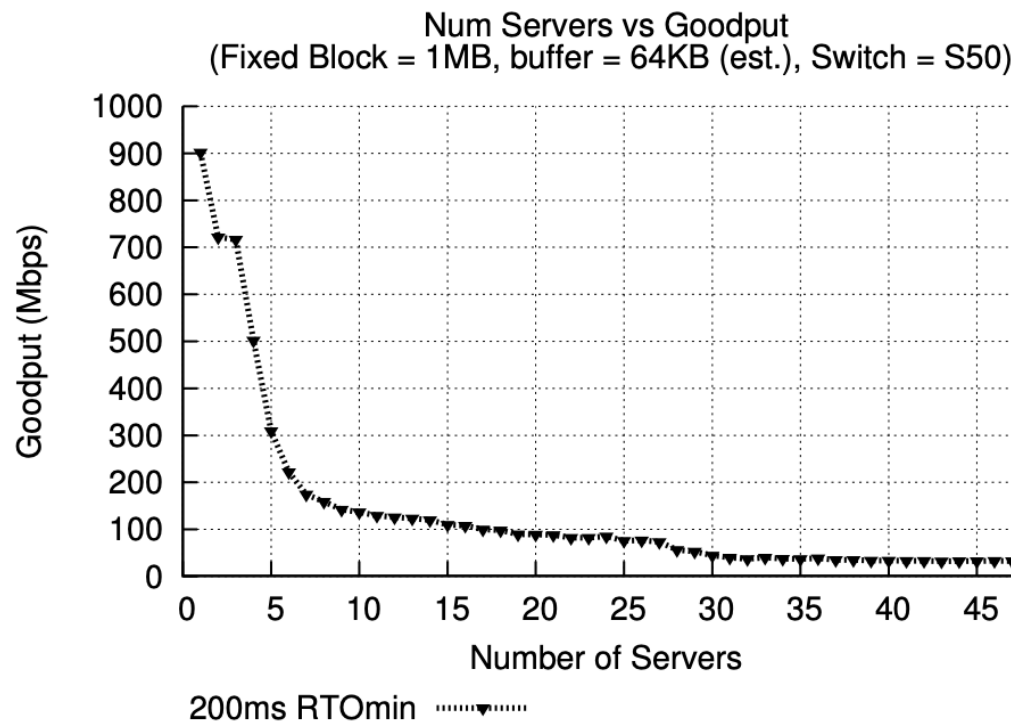
# This is what it is called TCP Incast



# This is what it is called TCP Incast



# This is what it is called TCP Incast



Food for thoughts for researchers. We will see more about this

# Recap on Datacenter characteristics

- Huge scale (order of 100K switches/routers)
- Limited geographic scope
- Limited heterogeneity
- Regular/planned topologies



# Recap on Datacenters today

- Very low latency within the datacenter (microsecond scale)
  - This impact buffering and switch design
  - This impact congestion control
- Need for high-bisection bandwidth
  - This impact routing
- Need for programmatic control
  - “you will never see traffic from Z”
  - “your VM will always have at least X bandwidth available”
- Care about cost/efficiency

# The Google answer to the problem (in 2015)

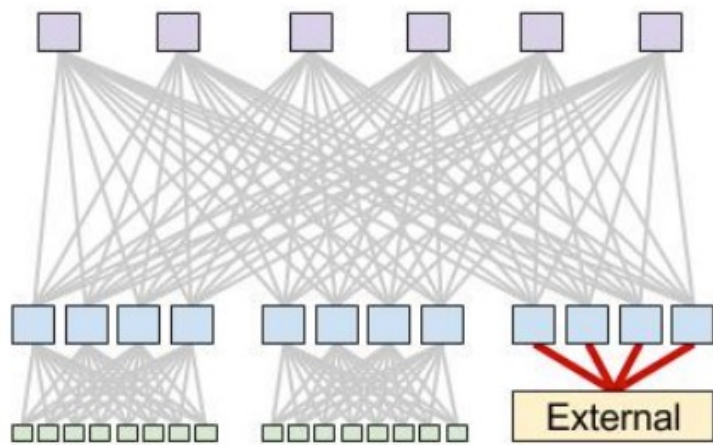
- Clos topology to support graceful fault tolerance and increase the scale/bisection of the network
  - Good for scaling: need to add stages to the topology
  - Good for failure recovery: there is substantial in-built path diversity and redundancy that a failure result in a small capacity reduction
- Use commodity merchant silicon rather than use commercial switches
  - No need for large feature sets
  - Frequent refresh cycles to keep up with bandwidth demands

# The Google answer to the problem (in 2015)

- Centralized control protocols
  - Control and management becomes substantially more complex with Clos topologies as the number of switching elements increases.
  - Existing routing and management protocols were not well-suited to such an environment
  - Idea: global configuration of predefined cluster plan proactively pushed on all switches

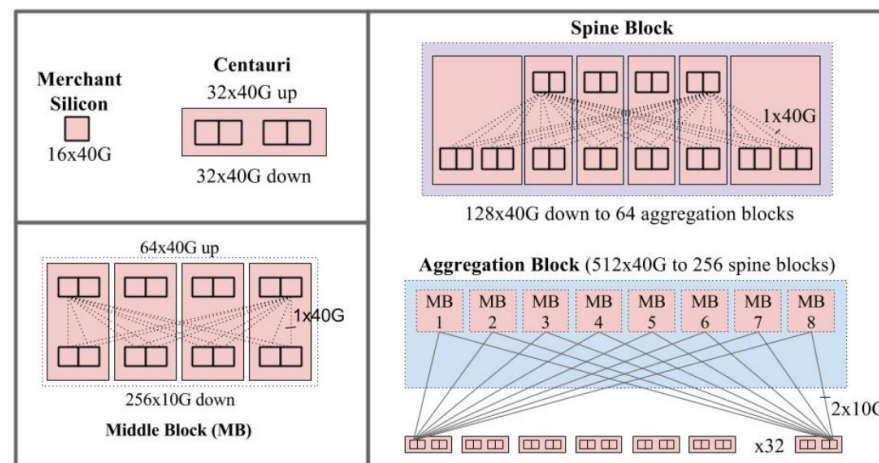
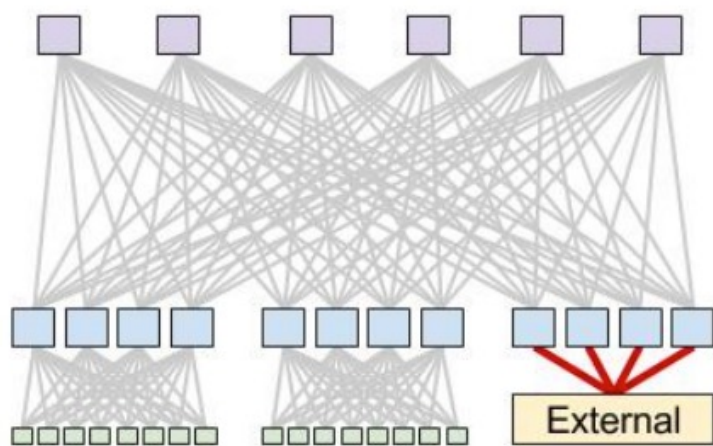
You will learn more about centralized control in the next lectures! 😊

# The Google answer to the problem (in 2015)

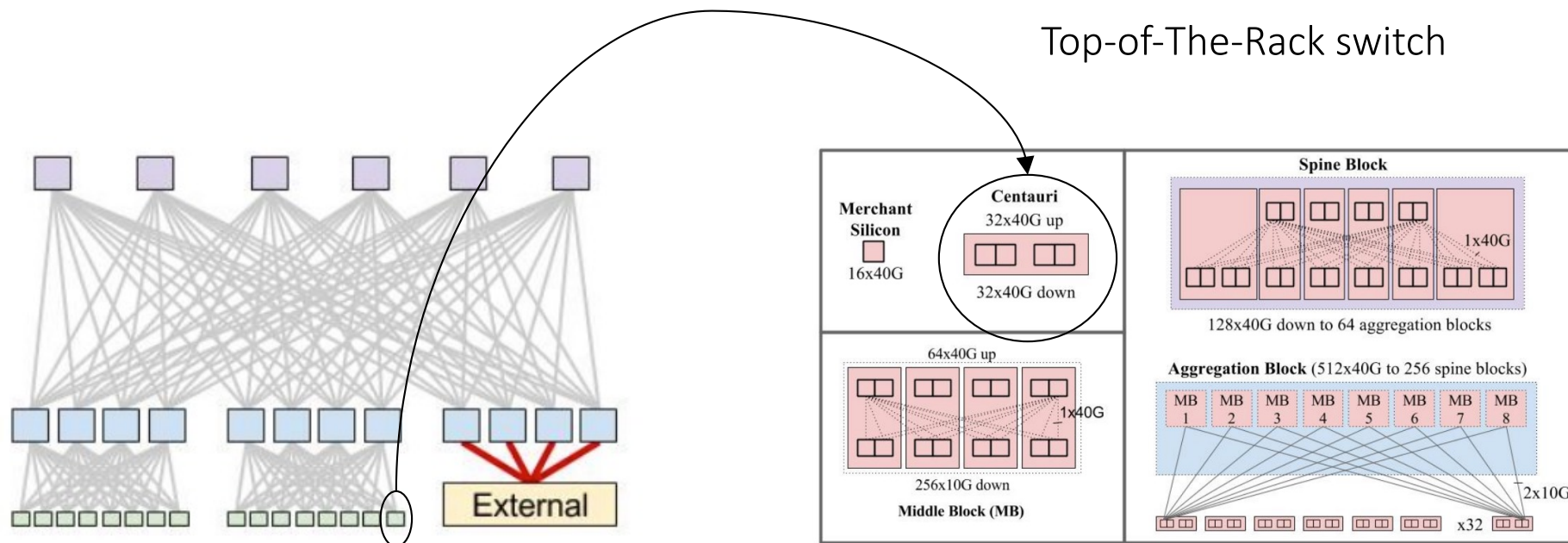


Google Jupiter Fabric

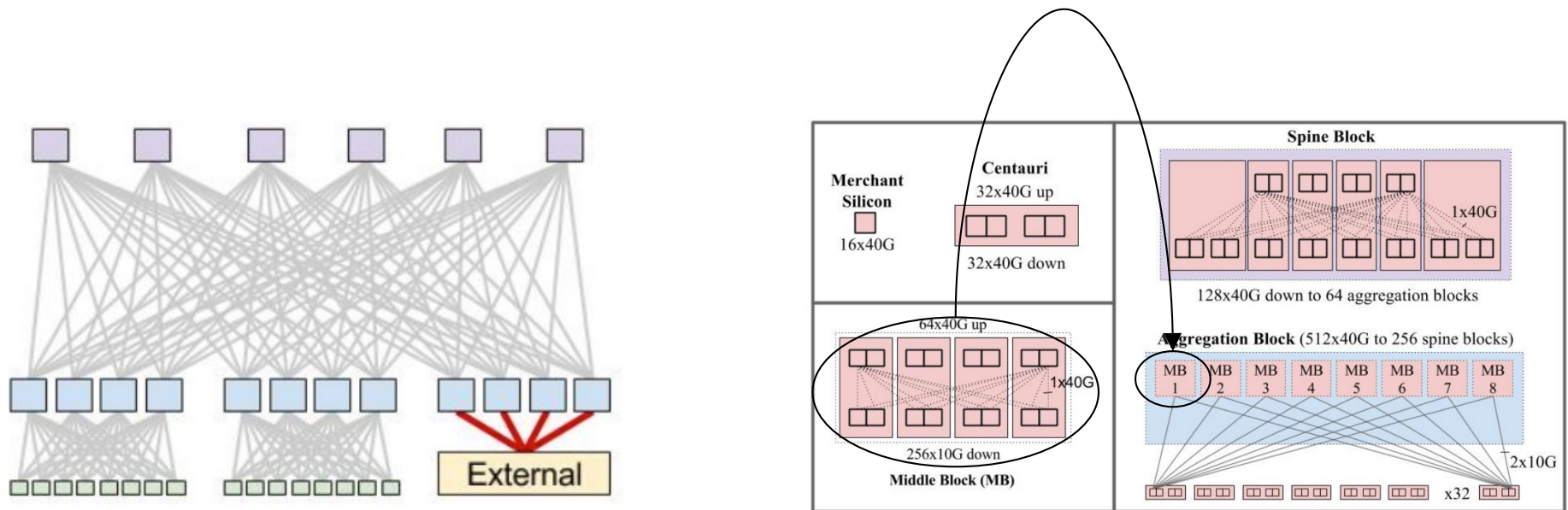
# The Google answer to the problem (in 2015)



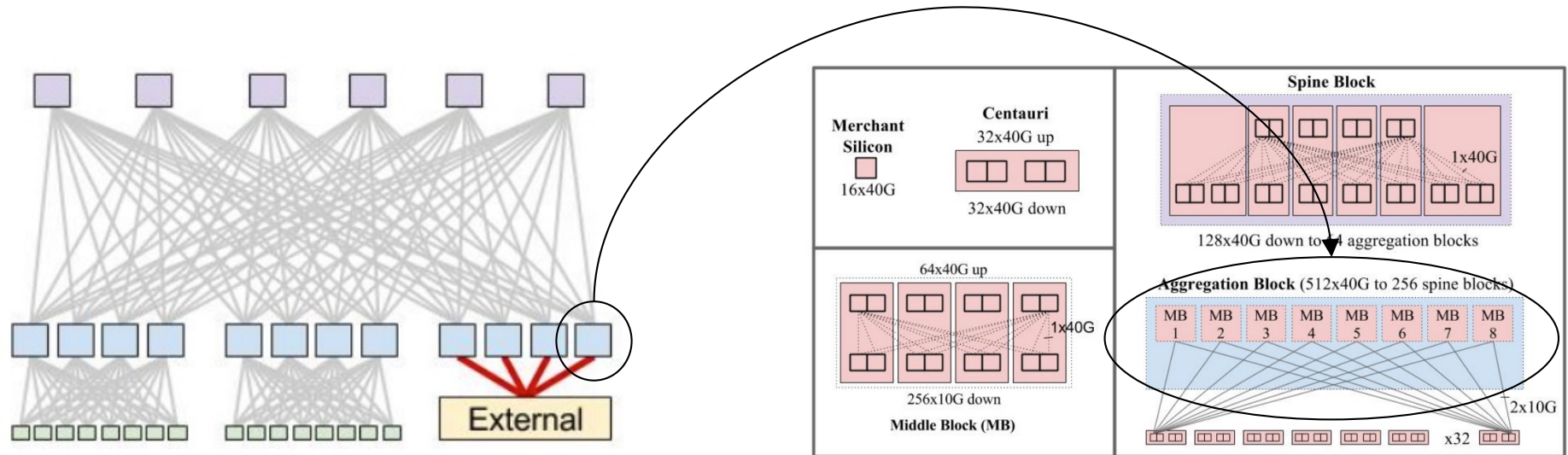
# The Google answer to the problem (in 2015)



# The Google answer to the problem (in 2015)

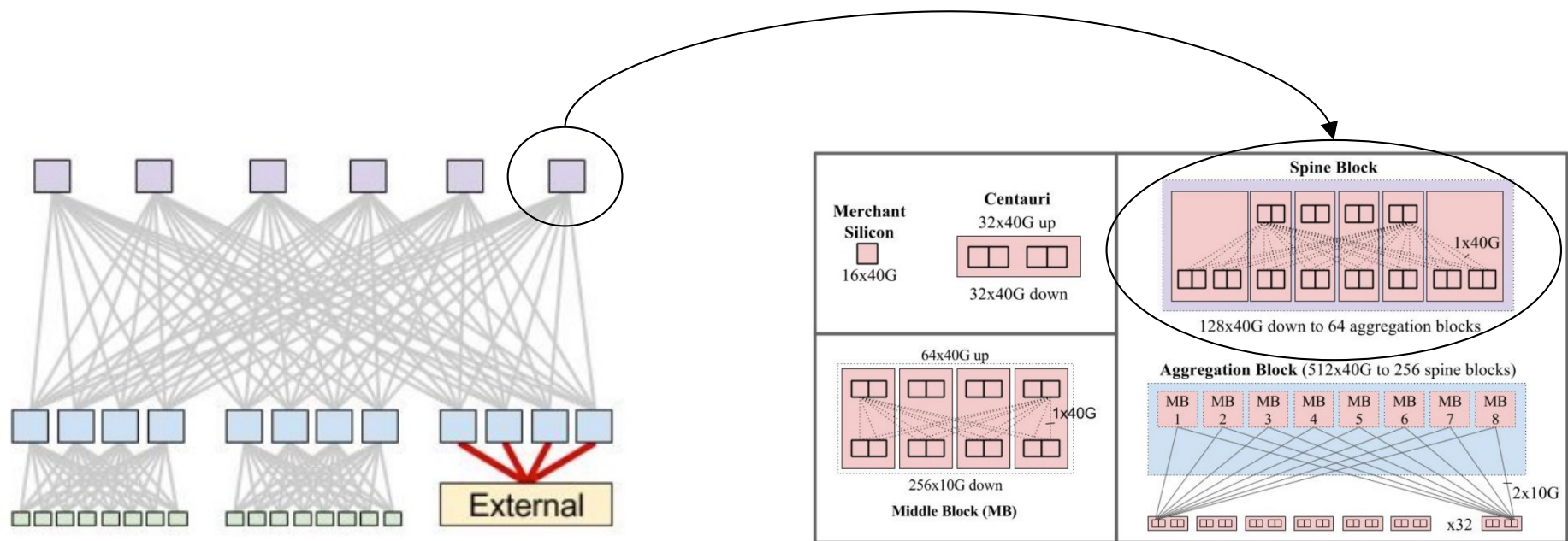


# The Google answer to the problem (in 2015)





# The Google answer to the problem (in 2015)

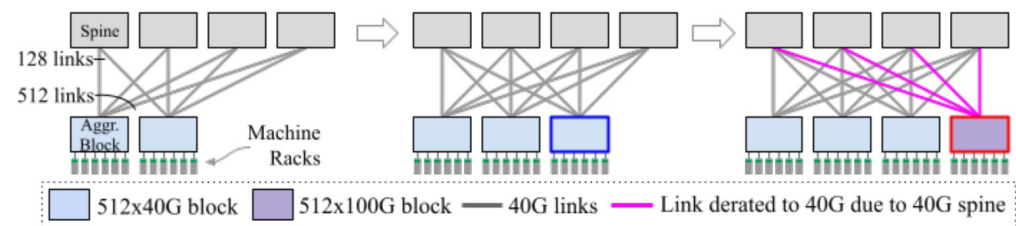


# The Google answer to the problem (now)

- In a recent paper, Google has announced its new datacenter network built for evolvability and performance

## Jupiter Evolving: Transforming Google's Datacenter Network via Optical Circuit Switches and Software-Defined Networking

Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq,  
Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble,  
Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj,  
Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura,  
Shidong Zhang, Junlan Zhou, Amin Vahdat  
Google  
sigcomm-jupiter-evolving@google.com



All spines are deployed on Day 1. Blocks deployed on Day 2 are outlined in blue while those deployed on Day N are highlighted in red. Links from a 100Gbps aggregation block are derated to 40Gbps due to the 40Gbps spine

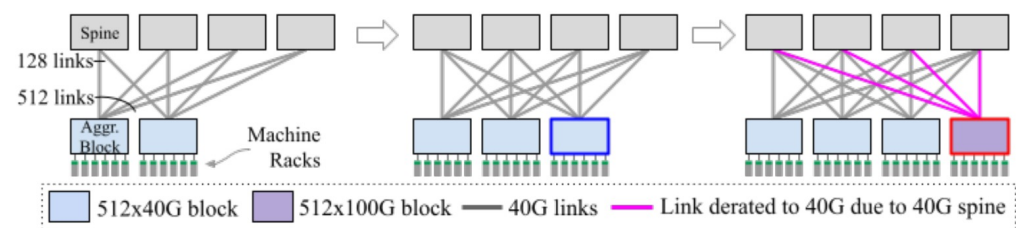
# The Google answer to the problem (now)

- In a recent paper, Google has announced its new datacenter network built for evolvability and performance

## Jupiter Evolving: Transforming Google's Datacenter Network via Optical Circuit Switches and Software-Defined Networking

Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beauregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzer, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, Amin Vahdat  
Google  
sigcomm-jupiter-evolving@google.com

The paper has lots of interesting insights!

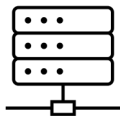


All spines are deployed on Day 1. Blocks deployed on Day 2 are outlined in blue while those deployed on Day N are highlighted in red. Links from a 100Gbps aggregation block are derated to 40Gbps due to the 40Gbps spine

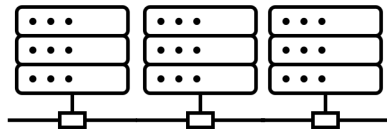
# What if a Datacenter is not enough?



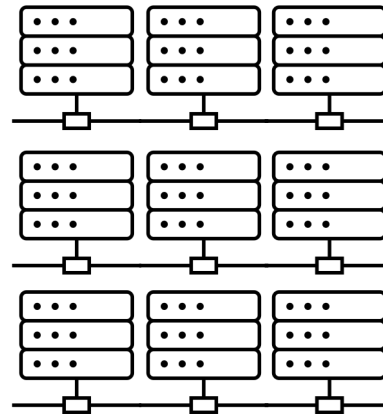
PC



Server

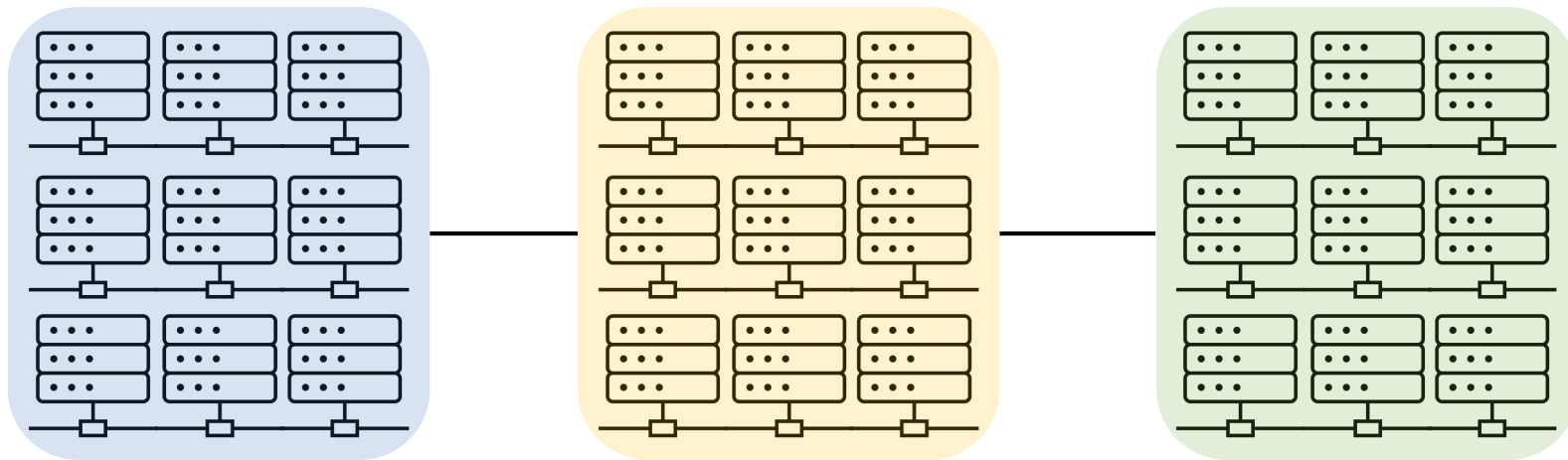


Cluster



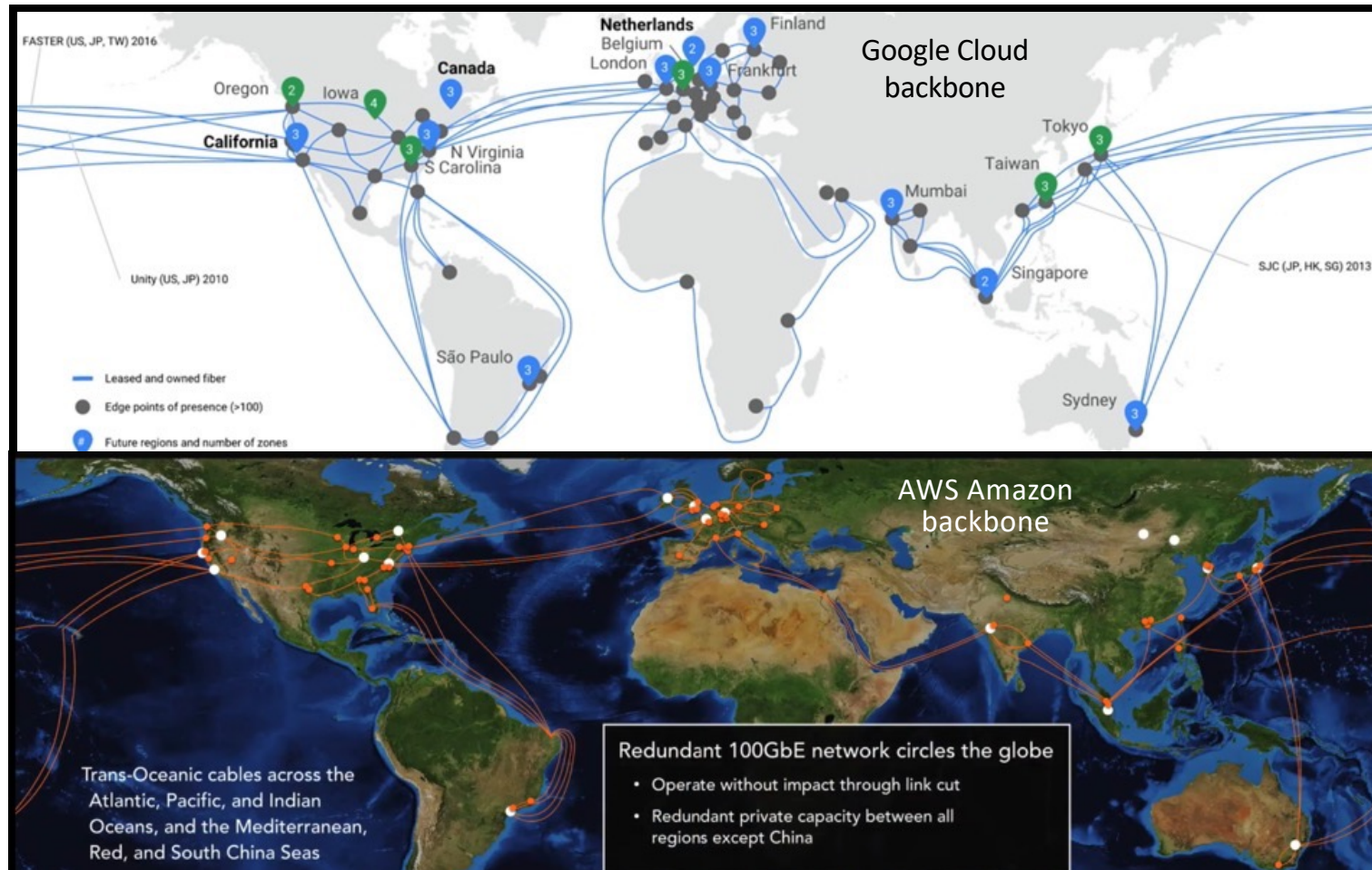
Datacenter

# What if a Datacenter is not enough?

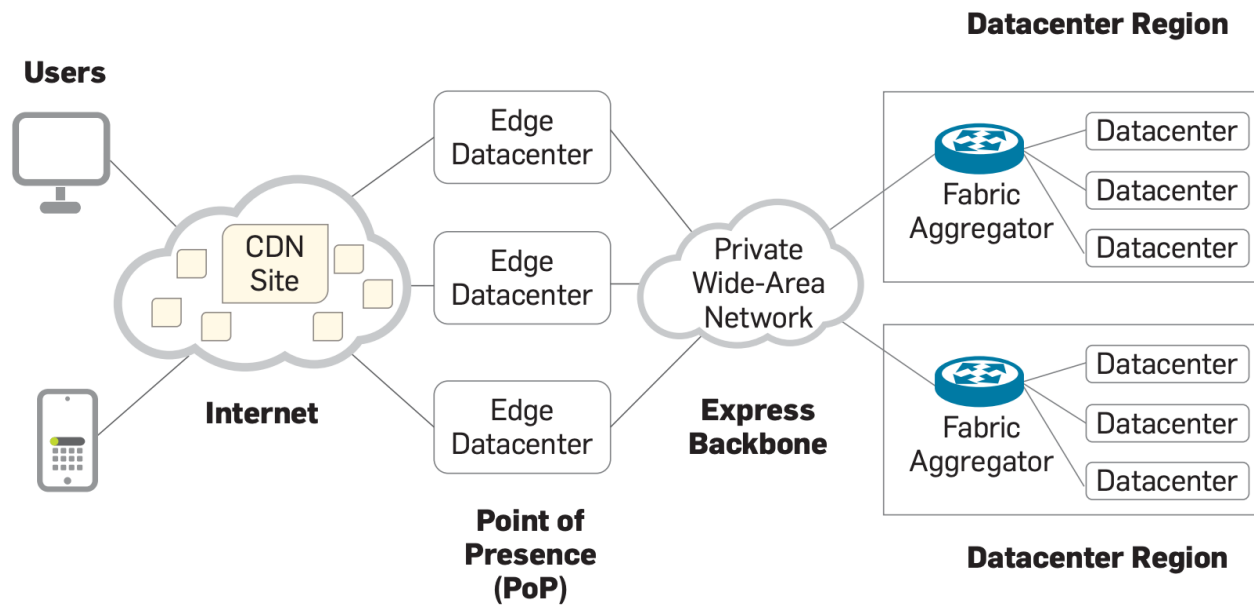


A network of datacenters!

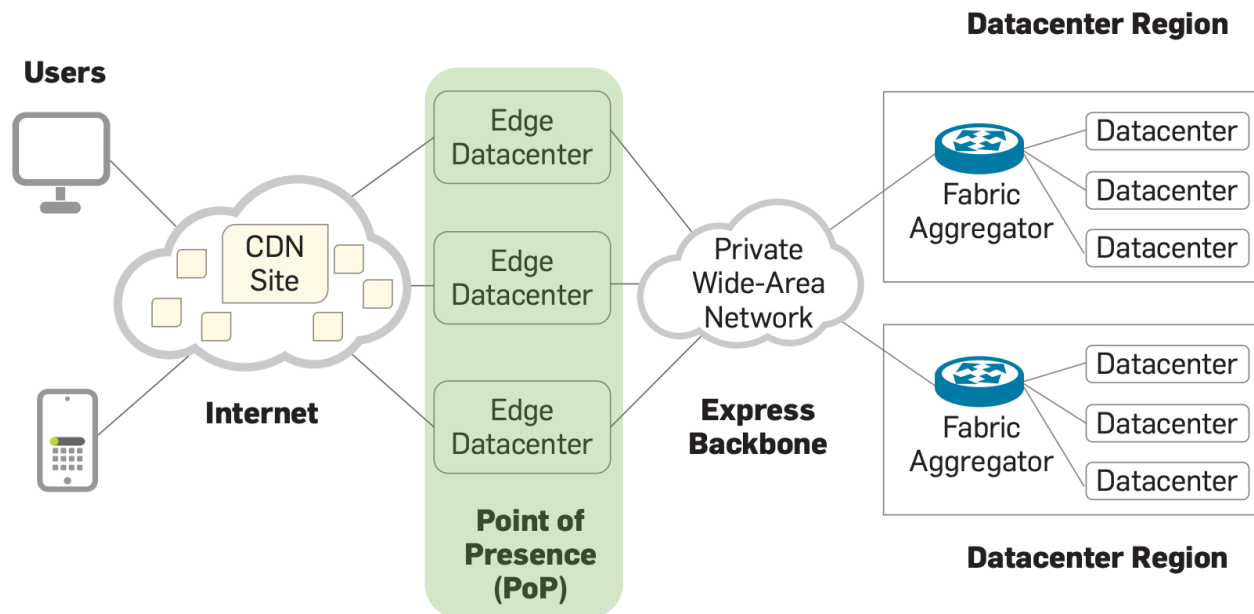
# Indeed, datacenters are geographically distributed



# A real-world example: Meta's Infrastructure



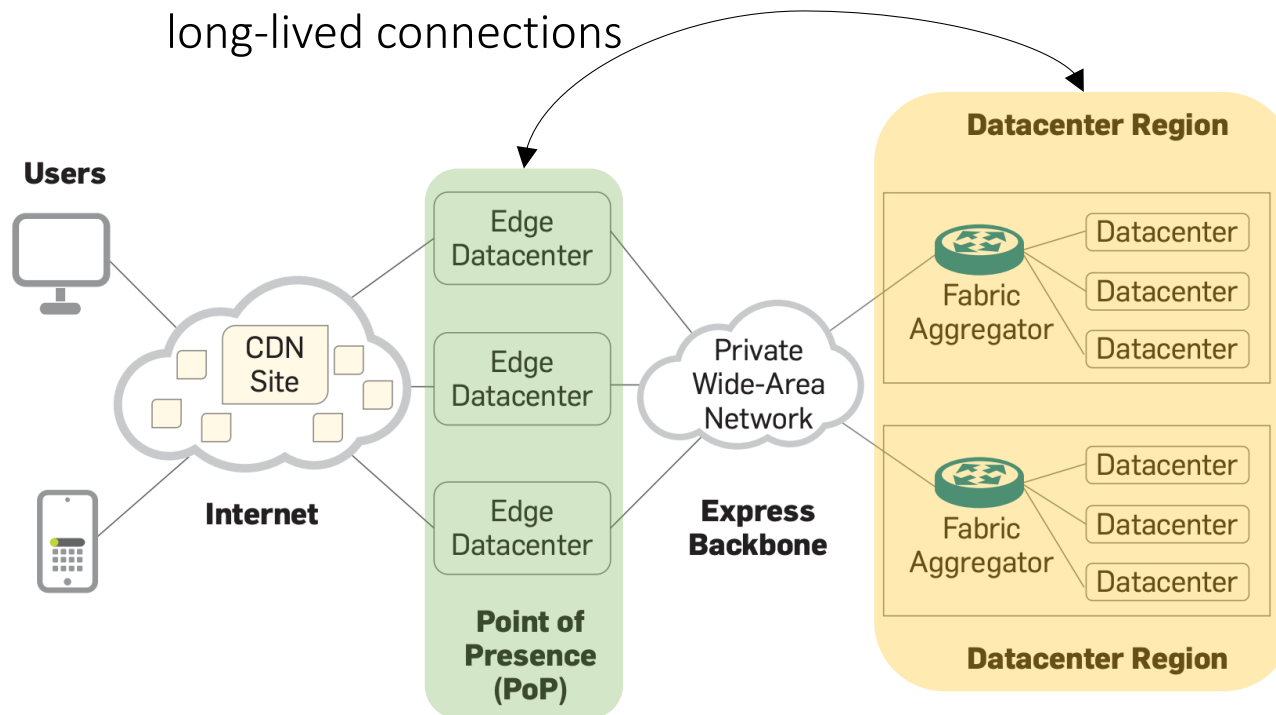
# A real-world example: Meta's Infrastructure



Users TCP/TLS connection termination

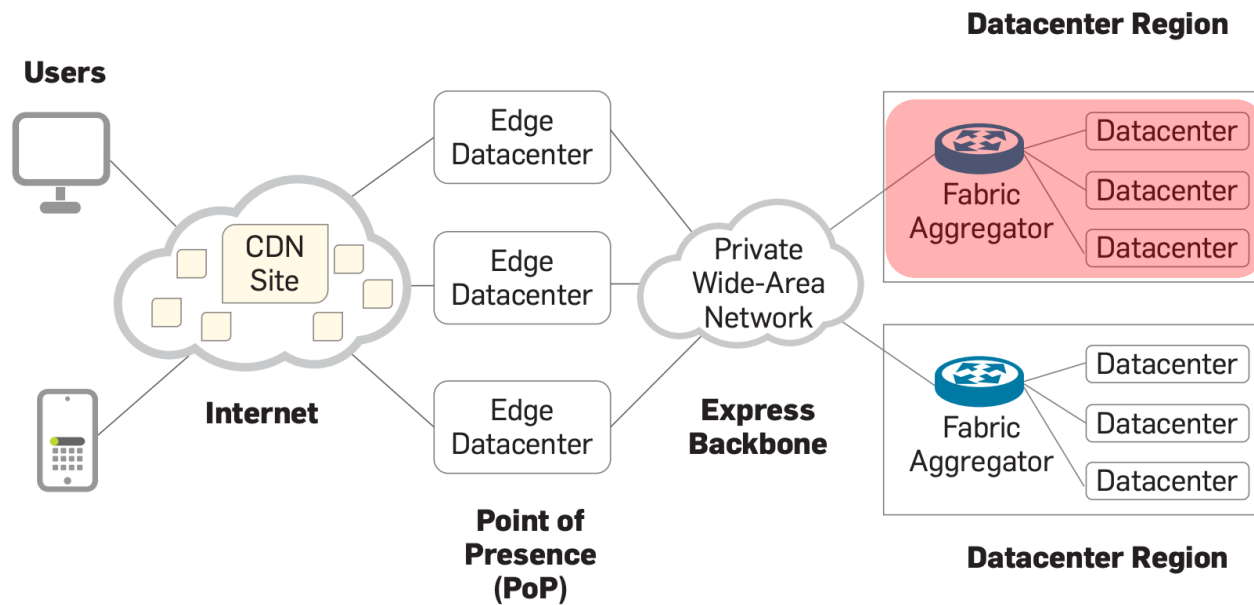


# A real-world example: Meta's Infrastructure



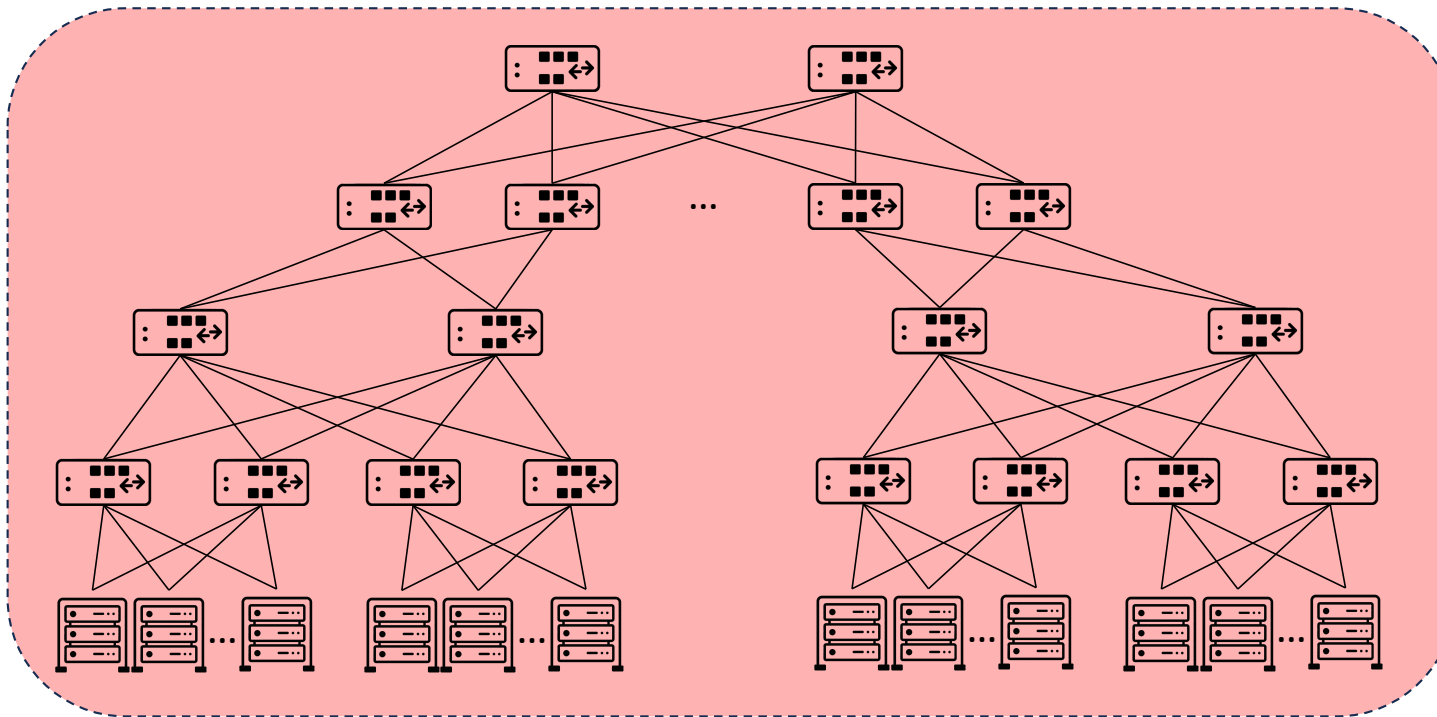
Users TCP/TLS connection termination

# A real-world example: Meta's Infrastructure



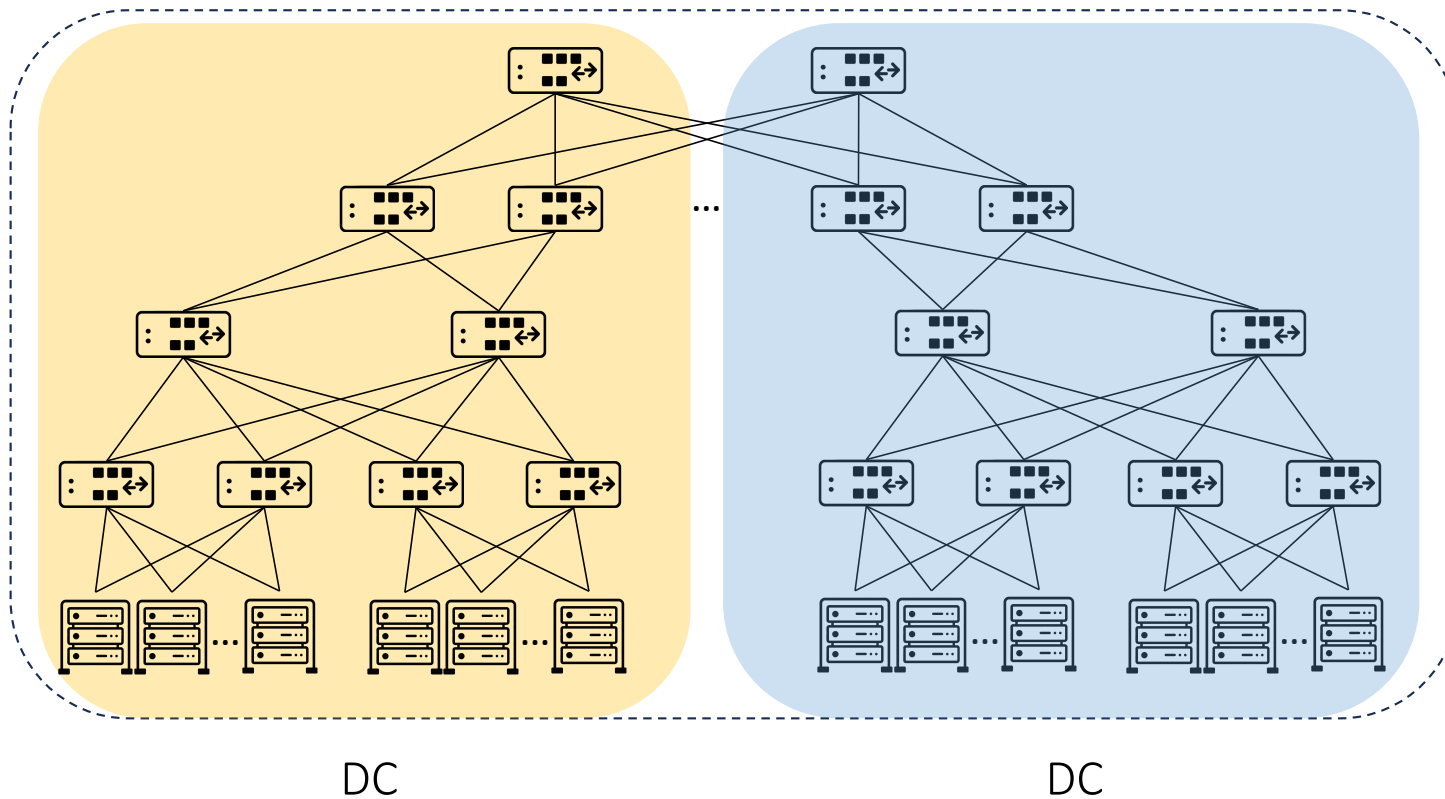
# Regional Data Center Interconnect

Datacenter  
region

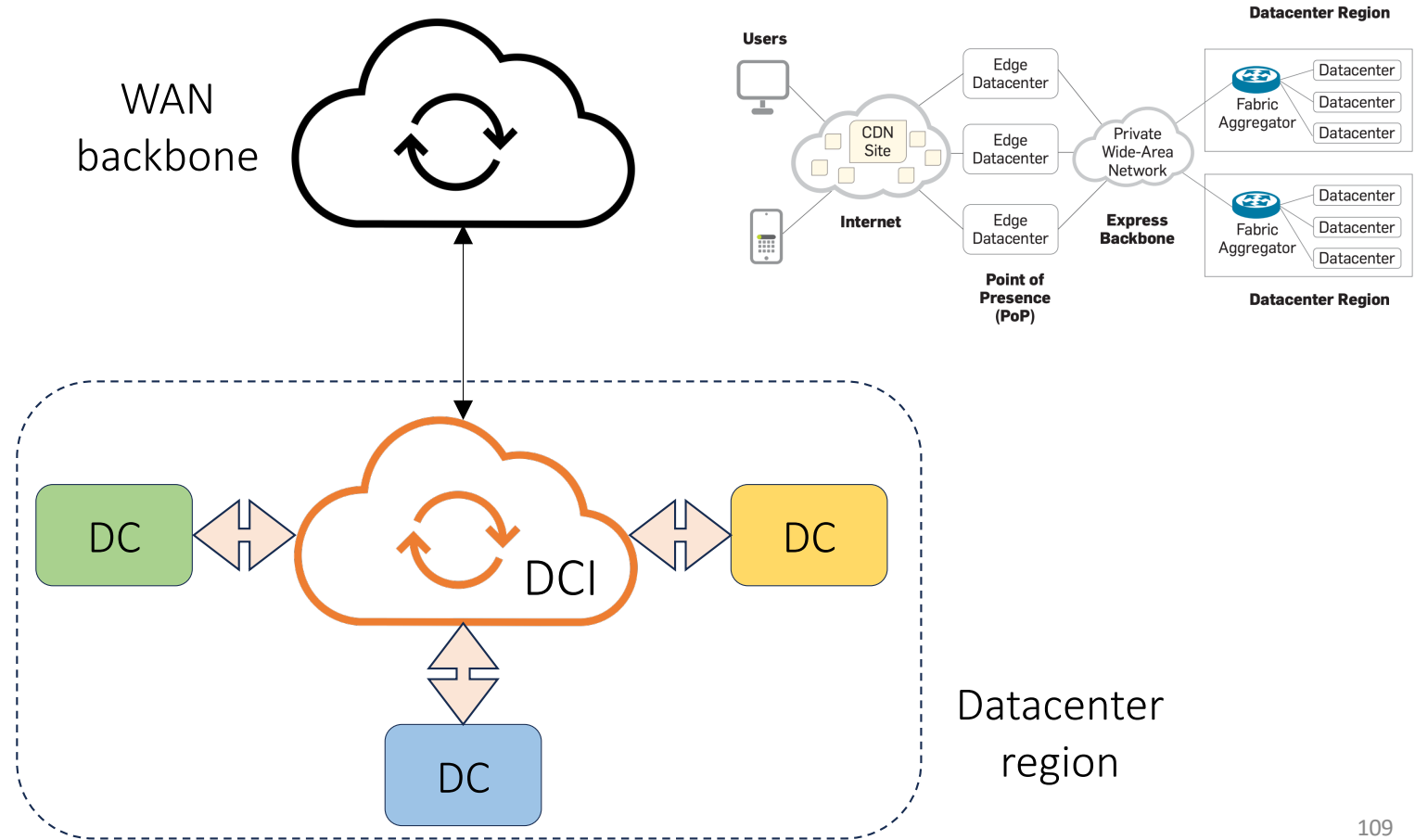


# Regional Data Center Interconnect

Datacenter  
region



# Regional Data Center Interconnect



# Fun facts

- Cooling is a very expensive problem

A research project to determine the feasibility of subsea datacenters



<https://natick.research.microsoft.com/>

# Fun facts

- Cooling is a very expensive problem
- Physical deployability matters, too!

*Where do racks fit?  
Do we have sufficient power and cooling?  
Can we make the cables long enough?*

## Physical Deployability Matters

Jeffrey C. Mogul  
Google

John Wilkes  
Google

### Abstract

While many network research papers address issues of deployability, with a few exceptions, this has been limited to protocol compatibility or switch-resource constraints, such as flow table sizes. We argue that good network designs must also consider the costs and complexities of deploying the design within the constraints of the physical environment in a datacenter: *physical* deployability. Traditional metrics of network “goodness” do not account for these costs and constraints, which might explain why some otherwise attractive designs have not been deployed in real-world datacenters.

We focus on datacenter networks, although physical-deployability issues certainly affect wireless networks [35] and WANs.

We explore whether deployability considerations might explain the non-deployment of otherwise attractive designs, such as expander graphs, in real-world datacenters. We also discuss technologies and techniques that would simplify physical deployment, and we speculate about the possibilities for devising well-specified objectives and metrics that could guide future research.

Designers of datacenter networks care about time-to-deploy, repair and maintenance processes, evolvability, and

Copper cables are cheaper & more reliable than fibers ... but they are fatter!

- There might not be enough room in the rack to fit enough of them
- They might fit, but block too much cooling airflow
- What if one does fail, and a technician has to replace it?

# Fun facts

(..) Sirius uses a combination of tunable lasers and simple, passive gratings that route light based on its wavelength (..) tightly codesigned with its routing and scheduling and with novel congestion-control (..) can offer high bandwidth and very low end-to-end latency

- Cooling is a very expensive problem
- Physical deployability matters, too!
- Optical is the present and future (?)

## Sirius: A Flat Datacenter Network with Nanosecond Optical Switching

Hitesh Ballani Paolo Costa Raphael Behrendt Daniel Cletheroe Istvan Haller  
Krzysztof Jozwik Fotini Karinou Sophie Lange Kai Shi Benn Thomsen Hugh Williams  
Microsoft Research

### ABSTRACT

The increasing gap between the growth of datacenter traffic and electrical switch capacity is expected to worsen due to the slowdown of Moore's law, motivating the need for a new switching technology for the post-Moore's law era that can meet the increasingly stringent requirements of hardware-driven cloud workloads. We propose Sirius, an optically-switched network for datacenters providing the abstraction of a single, high-radix switch that can connect thousands of nodes—racks or servers—in a datacenter while achieving nanosecond-granularity reconfiguration. At its core, Sirius uses a combination of tunable lasers and simple, passive gratings that route light based on its wavelength. Sirius' switching technology and topology is tightly codesigned with its routing and scheduling and with novel congestion-control and time-synchronization mechanisms to achieve a scalable yet flat network that can offer high bandwidth and very low end-to-end latency. Through a small-scale prototype using a custom tunable laser chip that can tune in less than 912 ps, we demonstrate 3.84 ns end-to-end reconfiguration atop 50 Gbps channels. Through large-scale simulations, we show that Sirius can approximate the performance of an ideal, electrically-switched non-blocking network with up to 74-77% lower power.

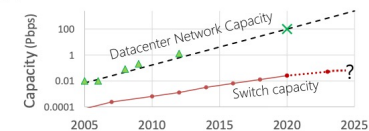


Figure 1: Datacenter network capacity (and traffic) [70] is growing faster than the doubling of switch capacity every two years (which, in turn, is expected to slowdown beyond 2024).

### 1 INTRODUCTION

The Moore's law for networking—electrical switches doubling their bandwidth every two years at the same power and cost—has allowed datacenter operators to scale up their network across generations. However, this impressive free scaling of switches still lags behind the doubling of cloud traffic roughly every year [70]. The increasing gap, shown in Fig. 1, results in high power, cost, and latency. For example, a large datacenter today would ideally be equipped with a 100 Pbps non-blocking network which would consume a prohibitive